

Informatik Projekt

im Studiengang

Informatik

Thema

EdgeML meets American Football

Vorgelegt von:

Gruppe1:

Amin Lahseb (1414161)

Ayman Lahseb (1432554)

Juan Fernando Lenis Serna (1438516)

Sahilpreet Singh (1513808)

Muhammad Naeem Abbas (1469530)

Arshad Reyat (1513428)

im Sommersemester 2026

am Fachbereich 2: Informatik und Ingenieurwissenschaften

der Frankfurt University of Applied Sciences

Prüfer: David Merkl, M.Sc.

Labor für eingebettete Systeme

Abgabedatum: 05.08.2026

Inhaltsverzeichnis

1. Einleitung	1
2. Methodik und Projektmanagement [Lenis Serna, Amin Lahseb]	2
2.1. Grundlagen und Framework-Auswahl	3
2.2. Rollenverteilung im Team	3
2.3. Scrum Master Retrospektive [Lenis Serna]	4
2.3.1. Umsetzung und Anpassung an das studentische Umfeld	4
2.3.2. Erläuterung und Vergleich der Scrum-Events	5
2.3.3. Hindernisse in der Praxis	5
2.3.4. Evaluation des Frameworks	6
2.4. Product Owner Retrospektive [Amin Lahseb]	7
2.4.1. Erstellung, Aufbau und Pflege des Product Backlogs	7
2.4.2. Zusammenarbeit mit Entwicklern und Scrum Master	8
2.4.3. Pflege Details und Refinement des Backlogs	9
2.4.4. Gegenüberstellung Theorie und Praxis	10
3. Teilprojekt	11
3.1. Jetson Nano [Sahilpreet Singh, Muhammad Naeem Abbas]	11
3.1.1. Einrichtung der Edge-Softwareumgebung	12
3.1.2. Modelloptimierung mittels TensorRT	12
3.1.3. Herausforderungen bei der Edge-Entwicklung	13
3.2. Hauptsteuerung und Video-Pipeline [Lenis Serna]	13
3.2.1. Asynchrone Bildverarbeitung	13
3.2.2. Evolution und Architektur des Hauptprogramms	15
3.2.3. Systemkonfiguration und Hardwareerkennung	19
3.3. Datenspeicherung Architektur [Ayman Lahseb]	20
3.3.1. Datenbank:	20
a. data_base.py:	20
b. data_class.py:	21
c. Main.py:	22
d. Heatmap.py:	23
3.4. Spielererkennung [Ayman und Amin Lahseb, Lenis Serna]	24
3.4.1. Spielerlokalisierung mit YOLO	24
3.4.1.1. Datenerhebung und Annotation	24
3.4.1.2. Modelltraining und Optimierung	26
3.4.1.3. Integration im Code und Pipeline-Ablauf	28
3.4.2. Trikotnummer Erkennung und Rollenzuweisung [Lenis Serna]	29
3.4.2.1. Erste Ansätze und Optimierung des Bildzuschnitts	29
3.4.2.2. Einführung eines Lesbarkeitsmodells	30
3.4.2.3. Asynchrone Verarbeitung und Caching-Strategie	31
3.4.3. Multi-Object Tracking und Re-Identifikation (Re-ID)	31
3.5. Erste Homographie durch verschiedene Ansätze [Arshad Reyah]	33

3.5.1. Hough Lines.....	33
3.5.2. White Pixel.....	35
3.5.3. Vanishing Points.....	37
3.5.4. Ausblick zur funktionsfähigen Homographie Berechnung/Erkennung.....	40
3.5.5. Erkenntnis.....	40
3.6. Homographie Engine [Ayman und Amin Lahseb, Lenis Serna].....	41
3.6.1. Erstes Training (1. Iteration).....	41
3.6.2. Zweites Training (2. Iteration).....	42
3.6.3. Drittes Training (3. Iteration).....	44
3.6.4. Integration und Optimierung der Homographie-Engine.....	45
3.7. Offensive Formationerkennung [Lenis Serna].....	47
3.7.1. Erste Versuche: Bildbasierte Klassifikation mit YOLO.....	47
3.7.2. Das Fehlen der räumlichen Perspektive.....	47
3.7.3. Der Durchbruch: Homographie und der Wechsel zu XGBoost.....	48
3.7.4. Datenbankstruktur und gezielte Datenaugmentation.....	48
3.7.5. Zeitliches Pooling zur Stabilisierung in der Inferenz.....	49
3.7.6. Ergebnis.....	50
3.8. Routen-Nachzeichnung [Amin Lahseb].....	50
3.8.1. Speicherung und Pflege der Laufkoordinaten.....	50
1. Bestimmung des Bodenkontaktpunkts.....	50
2. Perspektivische Transformation mittels Homographie.....	51
3. Persistierung und Aufbereitung der Trajektorien.....	51
3.1. Batch-Persistierung.....	51
3.2. Generierung der Trajektorienmatrix.....	52
3.8.2. Automatische Generierung von PlayCards.....	53
3.8.3. beispiel einer generierten Playcard.....	55
3.8.4. PlayCards-Benutzeroberfläche (GUI).....	56
3.9. Play Recognition [Ayman Lahseb].....	58
3.9.1. Modell Trainieren:.....	58
a. XGBoost (Extreme Gradient Boosting).....	58
b. YOLO.....	60
3.9.2. Anwendung:.....	61
3.10. Webanwendung und lokaler Webserver [Lenis Serna].....	62
3.10.1. Architektur und Technologiestack.....	62
3.10.2. Offline-Deployment als lokaler Hotspot.....	63
3.10.3. Benutzeroberfläche und Echtzeit-Dashboard.....	64
3.10.4. Limitationen und zukünftige Entwicklungen.....	67
4. Integration.....	68
5. Gesamtevaluation.....	70
6. Reflexion.....	72
Literaturverzeichnis.....	75
Eigenständigkeitserklärung / Urheberrechtshinweis.....	76

Abbildungsverzeichnis

Abbildung 1: Struktur des Product Backlogs in Google Sheets.....	8
Abbildung 2: Bildannotation für das YOLO-Modell in Roboflow.....	25
Abbildung 3 : Entwicklung der mAP@50-Metrik zur Spielererkennung (v1-v6).....	26
Abbildung 4: Verarbeitungsprozess der Python-Pipeline.....	28
Abbildung 5: Hough Lines Ansatz.....	35
Abbildung 6: White Pixel Ansatz.....	37
Abbildung 7: Erster Ansatz mit Mittellinie und Numbers.....	42
Abbildung 8: Zweite Version des Spielfeld-Skeletts mit Annotation.....	44
Abbildung 9: Zweite Version des Spielfeld-Skeletts mit Annotation.....	46
Abbildung 10: Finales Spielfeld-Skelett mit Annotation.....	47
Abbildung 11: PlayCard-Visualisierung (wide-clip151).....	57
Abbildung 12: Benutzeroberfläche des PlayCards Viewers.....	58
Abbildung 13: Hauptansicht der Webanwendung.....	67
Abbildung 14: Echtzeit-Analyse Dashboard (Latenzen & Ressourcen).....	68
Abbildung 15: UML-Sequenzdiagramm der Analyse-Pipeline.....	70
Abbildung 16: Pipeline-Leistungsdaten auf dem NVIDIA Jetson Nano.....	72

1. Einleitung

Die Automatisierung von Spielanalysen im Sport hat in den letzten Jahren durch Fortschritte im Bereich des Computer Vision und des maschinellen Lernens eine fundamentale Transformation erfahren. Während professionelle Sportorganisationen auf kostspielige Multi-Kamera-Systeme und cloudbasierte Serverinfrastrukturen zurückgreifen können, stehen Amateur- und Halbprofivereine vor der Herausforderung, mit begrenzten finanziellen und technischen Mitteln präzise taktische Einblicke zu gewinnen. Dies gilt insbesondere für komplexe Sportarten wie American Football, bei denen das Zusammenspiel von bis zu 22 Spielern auf dem Feld, schnelle Positionswechsel und eine Vielzahl spezifischer Pre-Snap-Formationen eine enorme Anforderung an die visuelle Erfassung stellen.

Die Motivation dieses Projekts liegt darin, diese Lücke durch die Entwicklung eines leichtgewichtigen, integrierten und ressourceneffizienten Analysesystems zu schließen. Das System soll in der Lage sein, standardmäßige Videomitschnitte von Spielen automatisiert zu verarbeiten, Spieler zu lokalisieren, deren Rollen und Trikotnummern zu erkennen sowie die gelaufenen Routen und Formationen der Offense zu rekonstruieren. Ein zentraler Anreiz besteht darin, die Inferenz und Datenverarbeitung nicht zwingend an teure Rechenzentren zu binden, sondern auch die Ausführung auf mobiler Edge-Hardware wie dem NVIDIA Jetson Nano zu ermöglichen. Dadurch wird Trainern und Anwendungsnutzern die Möglichkeit geboten, fundierte visuelle Taktik-Playcards und Laufanalysen direkt vor Ort und mit minimaler Verzögerung zu generieren.

Aus dieser Problemstellung ergibt sich die zentrale Forschungsfrage dieser Arbeit: Inwieweit lässt sich eine mehrstufige Computer-Vision-Pipeline zur automatisierten Analyse von Football-Formationen und Trajektorien so konzipieren und optimieren, dass sie auf ressourcenbeschränkter Edge-Hardware echtzeitnahe Ergebnisse liefert, ohne dabei signifikante Einbußen bei der Erkennungsgenauigkeit in Kauf nehmen zu müssen?

Zur Beantwortung dieser Fragestellung ist das primäre Ziel der Arbeit die Konzeption, Implementierung und Evaluation einer modular aufgebauten Software-Pipeline. Diese führt verschiedene KI-Komponenten zusammen, darunter ein YOLO-basiertes Objekterkennungsmodell für die Spielerlokalisierung, eine Homographie-Engine zur Projektion von Bild- in reale Feldkoordinaten, eine optische Zeichenerkennung (OCR) zur Erfassung von Trikotnummern sowie ein XGBoost-Modell zur Klassifikation von Offensiv-Formationen. Ergänzt wird die Pipeline durch Mechanismen zur automatischen Playcard-Erstellung und eine Benutzeroberfläche zur Durchmusterung der Ergebnisse.

Zur klaren Abgrenzung des Projektumfangs liegt der Fokus der vorliegenden Arbeit primär auf der präzisen Erfassung und Analyse der Offensiv-Einheit sowie der Pre-Snap-Phasen vor dem Spielzug. Die Erfassung von Defensivspielern dient im aktuellen Entwicklungsstand vorrangig der visuell-räumlichen Abgrenzung zur Vermeidung von Fehlklassifikationen, während ein durchgehendes Tracking der Defense, ein dediziertes Ball-Tracking sowie komplexe Einzelspieler-Statistiken über das gesamte Spiel hinweg als zukünftige Erweiterungsstufen definiert sind und nicht Teil dieses Kernsystems bilden.

2. Methodik und Projektmanagement [Lenis Serna, Amin Lahseb]

Dieses Kapitel beleuchtet die Methodik und die organisatorischen Maßnahmen, die im Rahmen dieses Projekts angewandt wurden. Gemäß der Lehrveranstaltung war die Anwendung des agilen Frameworks Scrum als kollaborative Arbeitsgrundlage für die Teamarbeit verpflichtend vorgesehen. Jedoch, da es sich um ein studentisches Projekt handelt, musste das vorgegebene Scrum Framework an die spezifischen Umstände und Herausforderungen angepasst werden. Im Folgenden werden die erwähnten Anpassungen dargelegt. Anschließend erfolgt eine kritische Reflexion des Projektverlaufs aus Sicht der Scrum-Master und Product-Owner.

2.1. Grundlagen und Framework-Auswahl

Obwohl die Wahl des Frameworks durch die Projektrichtlinien auf Scrum festgelegt war, bot dieser Ansatz theoretisch die geeignete Basis, um die komplexe Entwicklung eines Edge-KI-Systems iterativ und inkrementell zu steuern. Die gesamte Projektlaufzeit von zehn Wochen wurde strukturell in fünf aufeinanderfolgende Sprints von jeweils zwei Wochen unterteilt. Die Dauer der Sprints wurde so gewählt, um einerseits ausreichend Bearbeitungszeit für komplexe Entwicklungsaufgaben zu gewährleisten und andererseits kurze Feedbackschleifen zur Fehlerbehebung zu etablieren.

Zur technischen Unterstützung und Nachverfolgbarkeit des Projektes wurde GitLab als zentrales Tool eingesetzt. Hierbei diente das Issue-Board der visuellen Verwaltung und Nachverfolgung des Produkt- und Sprint-Backlogs. Begleitend wurden Microsoft Teams für die Kommunikation und Meeting-Durchführung sowie Google Docs zur ausführlicheren Dokumentation der Sprints genutzt, um auch Teammitgliedern ohne tiefe GitLab-Erfahrung einen Zugang zu den Projektzielen zu ermöglichen.

2.2. Rollenverteilung im Team

Ein zentraler Bestandteil von Scrum ist die klare Zuweisung von Verantwortlichkeiten durch spezifische Rollen (Scrum-Master, Product-Owner und Developer). Das Projektteam bestand insgesamt aus sechs Mitgliedern. In der initialen Planungsphase zeigte sich jedoch bei der Mehrheit der Teammitglieder eine starke Zurückhaltung gegenüber den Management-Rollen. Dies war primär auf Unsicherheiten bezüglich des genauen Aufgabenprofils sowie auf Bedenken hinsichtlich des administrativen Mehraufwands und der damit verbundenen Verantwortung zurückzuführen.

Um einen organisatorischen Stillstand zu verhindern und den pünktlichen Projektstart zu garantieren, mussten diese Schlüsselrollen proaktiv aus dem engagierten Kern des Teams besetzt werden, so übernahm Herr Lenis Serna die Rolle des Scrum Masters, während Herr Amin Lahseb als Product Owner agierte.

Die verbleibenden vier Mitglieder wurden als Developer in das Team integriert, mit dem theoretischen Ziel, die im Backlog definierten Entwicklungsaufgaben eigenverantwortlich umzusetzen. Diese initiale Dynamik bei der Rollenvergabe war bereits ein erster Indikator für die spätere Herausforderung, Eigenverantwortung im gesamten Team zu etablieren.

2.3. Scrum Master Retrospektive [Lenis Serna]

2.3.1. Umsetzung und Anpassung an das studentische Umfeld

Um das Projekt strukturiert und iterativ durchzuführen, wurde ein zehnwöchiger Zeitplan entworfen, der in fünf Sprints von jeweils zwei Wochen unterteilt war. Diese Dauer wurde gewählt, um ausreichend Zeit für die Bearbeitung von Backlog-Items zu gewährleisten und gleichzeitig kurz genug zu sein, um auf technische Schwierigkeiten zeitnah reagieren zu können. Für das Aufgabenmanagement und die transparente Visualisierung der Arbeitsabläufe wurde ein Issue-Board in GitLab eingerichtet, unterstützt durch Google Docs als zusätzliche Dokumentation.

Es zeigte sich schnell, dass sich die strengen Regeln des Scrum-Guides im studentischen Projektalltag nicht 1:1 umsetzen ließen. Ein tägliches "Daily Scrum" von 15 Minuten war organisatorisch nicht machbar, da Nebenjobs und Nachholklausuren die gemeinsame Terminfindung erschwerten. Um dieses Format an die Realität des Teams anzupassen, wurde das Event modifiziert, sodass anstatt des täglichen Termins regelmäßige "Sync-Meetings" eingeführt wurden. Diese fanden zweimal pro Woche über MS Teams statt und waren auf eine feste Dauer von etwa 45 Minuten angesetzt. Auf diese Weise konnte ein verlässlicher Kommunikationsfluss sichergestellt werden, ohne den zeitlichen Rahmen der einzelnen Teammitglieder zu sprengen.

2.3.2. Erläuterung und Vergleich der Scrum-Events

Zu Beginn des Projektes wurden die initialen Sprints planmäßig mit einem Sprint Planning gestartet, bei dem die Aufgaben im GitLab-Board definiert wurden. In der Scrum-Theorie organisiert sich ein cross-funktionales Team selbst und zieht sich eigenverantwortlich Aufgaben aus dem Sprint Backlog (*Pull-Prinzip*). In der Praxis zeigte sich jedoch bereits im zweiten Sprint eine starke Passivität der Hälfte des Teams. Weder wurden Aufgaben selbstständig übernommen, noch wurde in den Sync-Meetings über Fortschritte gesprochen. Um den vollständigen Stillstand zu verhindern, musste von der Theorie abgewichen und ein *Push-Prinzip* angewandt werden, bei dem ich als Scrum Master den inaktiven Mitgliedern Aufgaben direkt zuwies.

Trotz dieser Maßnahme blieben die Ergebnisse aus. Da die formellen Scrum-Events ohne das Engagement des gesamten Teams keinen Mehrwert mehr boten, traf der aktive Kern des Teams eine radikale Entscheidung und zwar für den vierten und fünften Sprint wurden die formellen Scrum-Strukturen aufgelöst. Das Team wechselte in einen agilen Überlebensmodus (vergleichbar mit Kanban), bei dem die direkte Entwicklung kritischer Komponenten (z.B. Homographie und Modelltraining) durch den aktiven Kern priorisiert wurde, anstatt Zeit in unproduktive Planung von Meetings zu investieren.

2.3.3. Hindernisse in der Praxis

Das größte Hindernis im Projektverlauf war die mangelnde Eigenorganisation, die mäßige Kommunikation und das fehlende Commitment von drei der sechs Teammitglieder. Die ständigen Verzögerungen wurden häufig mit externen Verpflichtungen wie Nachholklausuren, Krankheit oder Arbeit begründet. Obwohl der aktive Kern des Teams denselben akademischen Belastungen ausgesetzt war, führte diese unfaire Einsatzbereitschaft dazu, dass die geplante Projektgeschwindigkeit drastisch einbrach.

Als Scrum Master versuchte ich, diese Blockaden zu lösen, indem ich die technische Einstiegshürde senkte. Anstatt Entwicklungsaufgaben zu delegieren, wurden den inaktiven Mitgliedern einfache, aber zeitaufwendige Aufgaben zugewiesen, wie das Annotieren von Bildern zur Verbesserung der Modelle. Trotz mehrfacher Erklärungen des Prozesses blieben auch diese Aufgaben lange Zeit unerledigt. Die Gefahren der asynchronen und unzuverlässigen Zusammenarbeit zeigten sich besonders deutlich kurz vor Projektende. Ein Teammitglied lieferte ein Paket von 200 annotierten Bildern erst sieben Tage vor der finalen Deadline. Bei der Überprüfung stellte sich heraus, dass die Daten redundante Clips enthielten, die der aktive Kern bereits verarbeitet hatte. Aufgrund der späten Lieferung war es unmöglich, diese Daten noch für ein Retraining der Modelle zu nutzen und ordnungsgemäß zu evaluieren. Die Ressourcen mussten verworfen werden, um die Stabilität des finalen Edge-ML-Systems nicht zu gefährden.

2.3.4. Evaluation des Frameworks

Schlecht: Die wesentliche Erkenntnis dieses Projekts ist, dass das Scrum-Framework intrinsische Motivation, ständige Verfügbarkeit und eine hohe Eigenverantwortung zwingend voraussetzt. Wenn diese Prämissen bei der Hälfte des Teams nicht gegeben sind, verwandelt sich Scrum von einem agilen Beschleuniger in administrativen Overhead. Die Pflege des Backlogs und das Abhalten von Meetings wurden zu einer bürokratischen Last, die den eigentlichen Entwicklungsfortschritt bremste.

Gut: Dennoch war der anfängliche Einsatz agiler Methoden nicht umsonst. Das agile Mindset in den ersten beiden Sprints ermöglichte es dem aktiven Kernteam, das Projekt frühzeitig in kritische technische Meilensteine zu zerlegen. Diese frühe Transparenz machte deutlich, dass essenzielle Aufgaben wie das Web-Hub (FastAPI/React), die Homographie-Pipelines und das Modelltraining gefährdet waren. Dadurch konnte das Kernteam die Verantwortung rechtzeitig übernehmen und die Entwicklung auf der Jetson-Hardware erfolgreich abschließen, anstatt am Ende des Semesters von unfertigen Aufgaben überrascht zu werden.

2.4. Product Owner Retrospektive [Amin Lahseb]

2.4.1. Erstellung, Aufbau und Pflege des Product Backlogs

Der erste Schritt eines erfolgreichen Projekts ist die Erstellung eines strukturierten Plans zum Ablauf der Arbeit, der Ziele und aller Zwischenschritte. Im Scrum-Framework sprechen wir von einem Product Backlog.

Nach dem Lesen der Projektbeschreibung und dem Erlangen einer grundsätzlichen Vorstellung vom Projekt und dessen Ablauf war der nächste Schritt, ein Product Backlog zu erstellen. Dieses zeichnet vorerst einen klaren Weg bis zum Ende des Projektes, kann jedoch jederzeit erweitert werden.

Das Product Backlog wurde in mehrere Epics unterteilt, die verschiedenen Hauptkategorien des Projekts zugeordnet sind. Zu jedem Epic gehören eine Aufgabe, eine Priorität (Hoch, Mittel, Niedrig) und ein Status („To Do“, „In Progress“, „Done“). Das gesamte Product Backlog wurde anschließend in ein Google Sheet übertragen, auf das jeder jederzeit zugreifen kann.

ID	Epic	Kategorie	Aufgabe / Item	Priorität	Status
PB-1.1	Epic 1: Edge-Infrastruktur	Hardware & Setup	Konfiguration des NVIDIA Jetson Nano und der Grafikkartentreiber für den ressourcenschonenden Offline-Betrieb.	Hoch	To Do
PB-1.2	Epic 1: Edge-Infrastruktur	Video-Pipeline	Entwicklung einer schnellen Verarbeitungspipeline, um vorhandene Spielvideos flüssig und in Echtzeit einzulesen.	Hoch	To Do
PB-1.3	Epic 1: Edge-Infrastruktur	Lokale Speicherung	Einrichtung einer einfachen, lokalen Datenhaltung (SQLite, JSON oder CSV), um erkannte Spieldaten direkt auf dem Gerät zu sichern.	Hoch	To Do
PB-1.4	Epic 1: Edge-Infrastruktur	Literatur & Recherche	Recherche, um Best Practices für sportliche Videoanalysen auf ressourcenbeschränkten Geräten anzuwenden.	Mittel	To Do
PB-2.1	Epic 2: Formationserkennung	Pre-Snap-Erkennung	Entwicklung einer Logik zur genauen Bestimmung des Video-Frames unmittelbar vor dem Start des Spielzugs.	Hoch	To Do
PB-2.2	Epic 2: Formationserkennung	Formations-Klassifikat	Auswahl eines leichten Detection-Modells, Training auf Football-Frames. Export nach TensorRT für Jetson. Validierung der Spielererkennung.	Hoch	To Do
PB-2.2	Epic 2: Formationserkennung	Modell-Integration	Recherche und Umsetzung eines CNN-basierten Klassifikationsmodells auf Basis des Papers aus MDPI Electronics (2023).	Hoch	To Do
PB-2.3	Epic 2: Formationserkennung	Trainingsdaten	Sammeln und Vorbereiten von passenden Bilddaten für das Training und die Validierung der Formationserkennung.	Hoch	To Do
PB-3.1	Epic 3: Spielererkennung & OCR	Video-Analyse	Untersuchung der Spielvideos auf Bildqualität, Lichtverhältnisse und Bewegungsunschärfe.	Hoch	To Do
PB-3.2	Epic 3: Spielererkennung & OCR	Spieler-Detektion	Training eines schnellen Objekterkennungsmodells, um die Umrisse (Bounding Boxes) der Spieler im Bild zuverlässig zu finden.	Mittel	To Do
PB-3.3	Epic 3: Spielererkennung & OCR	Texterkennung	Entwicklung und spezifische Optimierung eines OCR-Modells zum Auslesen der Nummern auf den Trikots.	Mittel	To Do
PB-3.4	Epic 3: Spielererkennung & OCR	Labeling	Markierung der Spieler mittels Bounding Boxes für das Detection-Training.	Hoch	To Do
PB-4.1	Epic 4: Routen-Nachzeichnung	Tracking	Auswahl und Konfiguration eines performanten Trackings (z. B. ByteTrack), um Wide Receiver über den gesamten Spielzug zu verfolgen.	Hoch	To Do
PB-4.2	Epic 4: Routen-Nachzeichnung	Kamera-Ausgleich	Rechnerische Korrektur von Kameraschwenks, damit die echten Laufwege trotz Bewegung der Kamera korrekt erfasst werden.	Hoch	To Do
PB-4.3	Epic 4: Routen-Nachzeichnung	Kurvenglättung	Mathematische Modellierung der rohen Tracking-Daten mittels Bernstein-Polynomen nach Chu et al. (2020).	Hoch	To Do
PB-4.4	Epic 4: Routen-Nachzeichnung	Parameter-Optimierung	Einsatz des EM-Algorithmus zur automatischen Anpassung der Kurvenmodelle an die realen Laufwege.	Hoch	To Do
PB-4.5	Epic 4: Routen-Nachzeichnung	Daten-Zwischenspeicherung	Kontinuierliches und performantes Schreiben der getrackten Koordinaten in lokale JSON-Dateien.	Hoch	To Do
PB-5.1	Epic 5: Homographie & Play Recog	Homographie	Umrechnung der Pixelkoordinaten aus dem Video auf ein flaches, statisches 2D-Spielfeldmodell.	Hoch	To Do
PB-5.2	Epic 5: Homographie & Play Recog	Spielfeldmarkierungen	Automatische Erkennung von Feldlinien zur genauen Berechnung der Perspektiven-Matrix.	Hoch	To Do
PB-5.3	Epic 5: Homographie & Play Recog	Datenzusammenführung	Aggregation der erkannten Formation, der getrackten Wege und der Routen in einem gemeinsamen Schritt.	Hoch	To Do
PB-5.4	Epic 5: Homographie & Play Recog	Spielzug-Bestimmung	Logik zur automatischen Klassifikation des final gespielten Angriffsspielzugs (Lauf- oder Passspielzug).	Hoch	To Do
PB-5.5	Epic 5: Homographie & Play Recog	Pipeline-Recherche	Analyse der Gesamt-Pipeline aus der Literatur zur "NFL Film Classification" für eine verlässliche Erkennungskette.	Mittel	To Do
PB-6.1	Epic 6: System-Integration	Schnittstellen definieren	Einfache Datenformate festlegen, damit die einzelnen Erkennungsmodule reibungslos Informationen in Python austauschen.	Hoch	To Do
PB-6.2	Epic 6: System-Integration	Modell-Optimierung	Umwandlung der trainierten Modelle (über das ONNX-Format) in schnelle TensorRT-Engines für maximale Inferenzgeschwindigkeit.	Hoch	To Do
PB-6.3	Epic 6: System-Integration	Hauptprogramm	Entwicklung der zentralen Steuerung, die das Video einliest und die sequenzielle oder parallele Ausführung der Teilmodelle koordiniert.	Hoch	To Do
PB-6.4	Epic 6: System-Integration	Ressourcen-Tests	Überprüfung der Systemstabilität und Auslastung (RAM, Temperatur und FPS) des Jetson Nano bei echten Videotests.	Hoch	To Do
PB-6.5	Epic 6: System-Integration	Agile Steuerung	Aufbau und Pflege des digitalen SCRUM-Boards sowie die Organisation und Moderation der regelmäßigen Sprint-Meetings.	Hoch	To Do
PB-7.1	Epic 7: Visualisierung & Coaching	Playcard-Zeichnung	Automatische Erstellung einer übersichtlichen 2D-Grafik (Playcard) für jeden erkannten Spielzug.	Hoch	To Do
PB-7.2	Epic 7: Visualisierung & Coaching	Routen-Darstellung	Einzeichnen der Startpositionen und der klassifizierten Standard-Routen mit klaren Symbolen auf der Playcard.	Hoch	To Do
PB-7.3	Epic 7: Visualisierung & Coaching	Datei-Export	Automatisches Speichern der fertigen Play Cards als Bilddateien (PNG) oder PDF-Dokumente.	Hoch	To Do
PB-7.4	Epic 7: Visualisierung & Coaching	Scouting-Report	Erstellung einer einfachen statistischen Übersicht der analysierten Spielzüge zur direkten Unterstützung des Coachings.	Mittel	To Do

Abbildung 1: Die Abbildung zeigt den tabellarischen Aufbau des Product Backlogs, das in Google Sheets strukturiert wurde. Der Tabellenkopf gliedert die Anforderungen in sechs übersichtliche Spalten

2.4.2. Zusammenarbeit mit Entwicklern und Scrum Master

Die Zusammenarbeit innerhalb des Teams zeigte sowohl Stärken in der Führungsebene als auch deutliche Herausforderungen im Entwicklungsbereich.

Die Interaktion mit dem Scrum Master verlief während des gesamten Projekts äußerst harmonisch und produktiv. Durch eine enge Abstimmung ergänzten sich Product Owner und Scrum Master gegenseitig: Erklärungen wurden gemeinsam vermittelt und organisatorische sowie inhaltliche Aufgaben hand in hand gelöst. Neben den offiziellen Terminen fanden regelmäßige zusätzliche Abstimmungen statt, um Meetings vorzubereiten, den Projektfortschritt zu evaluieren und die nächsten Schritte zu planen.

Im Gegensatz dazu stellte die Kommunikation mit dem Entwicklungsteam die größte Hürde des Projekts dar. Obwohl Aufgabenstellungen seitens des Product Owners verständlich aufbereitet wurden, gab es von Seiten der Entwickler kaum Rückfragen. Dies führte zu einer Ungewissheit darüber, ob die Anforderungen vollständig durchdrungen wurden. Erschwerend kam hinzu, dass die Teilnahme an den wöchentlichen Meetings über Microsoft Teams unvollständig war und regelmäßig fehlten zwei bis drei Entwickler. Da der Bearbeitungsstatus und die tatsächliche Aktivität einzelner Teammitglieder oft erst sehr spät transparent wurden, leistete etwa die Hälfte der Entwickler keinen Beitrag zum Projekt.

Hinsichtlich der Aufwandsschätzungen zeigte sich ein gespaltenes Bild: Während der Großteil der Aufgaben innerhalb des ersten einwöchigen Sprint-Zeitraums erfolgreich umgesetzt wurde, wurde der zeitliche Aufwand für das KI-Modell-Training drastisch unterschätzt. Dieses zog sich über die gesamte Projektlaufzeit hinweg. Diejenigen Entwickler, die aktiv am Projekt mitarbeiteten, erfüllten die fachlichen Erwartungen jedoch vollständig. Um den Projekterfolg nicht zu gefährden, mussten die ausgefallenen Aufgaben schlussendlich vom Product Owner und dem Scrum Master mitübernommen werden.

2.4.3. Pflege Details und Refinement des Backlogs

Das Product Backlog stellte das zentrale Steuerungsinstrument während der gesamten Projektlaufzeit dar. Die kontinuierliche Pflege und Detaillierung des Dokumentes (Backlog Refinement) folgte dabei einem dynamischen Rhythmus, der eng an die wöchentlichen Präsenzveranstaltungen gekoppelt war. Im Anschluss an den Unterricht sowie nach der Präsentation von Zwischenergebnissen und dem Erhalt von Feedback wurde das Backlog regelmäßig von Product Owner und Scrum Master aktualisiert.

Hinsichtlich des Detailgrads zeichnete sich zu Projektbeginn ab, dass die Backlog-Items zunächst relativ grob formuliert werden mussten. Erst im Zuge fortschreitender Recherchen zu den technischen Rahmenbedingungen konnten die Anforderungen schrittweise konkretisiert und in detaillierte Teilaufgaben zerlegt werden.

Für die operative Durchführung wurden die Aufgaben wöchentlich im Git-Repository den jeweiligen Entwicklern zugewiesen. Da das eigenständige Aktualisieren des Bearbeitungsstatus durch das Entwicklungsteam im Arbeitsalltag häufig vernachlässigt wurde, übernahmen der Product Owner und der Scrum Master die kontinuierliche Nachverfolgung und Statuspflege im zentralen Google Sheet.

Im Projektverlauf machten unvorhergesehene technische Hürden eine flexible Re-Priorisierung erforderlich. Während zu Beginn lediglich ein Modell zur Spielererkennung eingeplant war, stellte sich im Rahmen der Vertiefung heraus, dass zusätzliche KI-Modelle für Homographie, OCR und die Spielzug-Erkennung (Play Recognition) zwingend notwendig waren. Um ein Weiterarbeiten am Gesamtsystem zu ermöglichen, wurde das Training dieser grundlegenden Modelle kurzfristig auf die höchste Priorität gesetzt, während andere Aufgaben zurückgestellt wurden. Trotz des dadurch entstandenen Mehraufwands und zeitweiser Doppelarbeiten erwies sich das Backlog durch diese Anpassungsfähigkeit als verlässliches Instrument zur Projektsteuerung.

2.4.4. Gegenüberstellung Theorie und Praxis

Die Gegenüberstellung der theoretischen Vorgaben des Scrum-Frameworks mit der tatsächlichen Praxis im studentischen Projekt zeigt signifikante Unterschiede auf. Während die Theorie von idealen Rahmenbedingungen und klar abgegrenzten Rollen ausgeht, erforderte die Realität im Projektalltag eine hohe Flexibilität und das Eingehen von Kompromissen.

In der Theorie ist der Product Owner primär für die Maximierung des Wertes des Produkts sowie für das effektive Management des Product Backlogs verantwortlich. Er definiert Anforderungen (User Stories), priorisiert diese und nimmt die fertiggestellten Arbeitsergebnisse des Entwicklungsteams auf Basis von Akzeptanzkriterien und der Definition of Done ab. Das Entwicklungsteam agiert dabei selbstorganisiert und setzt die priorisierten Aufgaben eigenständig um, während der Scrum Master als Facilitator den Prozess schützt und Hindernisse beseitigt.

In der Praxis weichte die Rollentrennung stark auf. Bedingt durch die mangelnde Kommunikation und den Ausfall von Teilen des Entwicklungsteams konnte die im Scrum Guide vorgesehene Selbstorganisation des Teams nicht vollständig aufrechtgehalten werden. Die Aufgaben des Product Owners beschränkten sich daher nicht nur auf die reine Fach- und Prioritätenseite. Um den Projekterfolg und das Erreichen der Meilensteine sicherzustellen, mussten Product Owner und Scrum Master als eng kooperierendes Führungs-Duo agieren. Dies führte dazu, dass der Product Owner nicht nur die Statuspflege im Backlog und Git-Repository übernehmen, sondern auch aktiv bei der technischen Umsetzung und der Übernahme liegengelassener Entwicklungsaufgaben mitwirken musste.

Zusätzlich zeigte sich in der Praxis ein Unterschied bezüglich des Backlog Refinements und der Aufwandsschätzung. Theoretisch liefert das Entwicklungsteam das technische Know-how zur Aufwandseinschätzung im Sprint Planning. Praktisch führte die Fehleinschätzung des Modelltrainings dazu, dass der Product Owner die Prioritäten im Backlog kurzfristig und ad hoc umstrukturieren musste, um den Projektfluss aufrechtzuerhalten.

Zusammenfassend zeigte sich, dass Scrum eine gute Orientierung bot. In der Praxis mussten wir die festen Rollen jedoch lockerer handhaben, um uns gegenseitig zu helfen und das Projekt voranzubringen.

3. Teilprojekt

3.1. Jetson Nano [Sahilpreet Singh, Muhammad Naeem Abbas]

Damit das entwickelte System nicht nur auf leistungsstarker Server- oder Laptop-Hardware, sondern auch direkt am Spielfeldrand einsetzbar ist, wurde die gesamte Pipeline zusätzlich auf Edge-Hardware der NVIDIA-Jetson-Familie portiert und für den ressourcenbeschränkten Betrieb optimiert.

3.1.1. Einrichtung der Edge-Softwareumgebung

Für die Edge-Bereitstellung haben wir uns letztlich für das offizielle JetPack-5-Image von NVIDIA entschieden, weil CUDA, cuDNN und TensorRT dort schon vorkonfiguriert sind. Ursprünglich war geplant, alle Abhängigkeiten manuell herunterzuladen und selbst in der richtigen Versionskombination zu installieren. Das haben wir dann aber verworfen. Der Grund: GPU-Treiber, CUDA und cuDNN hängen auf Jetson-Hardware sehr eng und genau dokumentiert voneinander ab, und schon kleine Versionsunterschiede sorgten für Inkompatibilitäten, die sich kaum noch zurückverfolgen ließen. Mit dem vorkonfigurierten JetPack-Image fiel dieser ganze Ärger weitgehend weg.

Ein weiteres Problem tauchte bei OpenCV auf. Die Version aus dem Standard-Paketmanager war nur ein headless-Build, also ohne GUI. Für die lokale Visualisierung der Analyseergebnisse brauchten wir aber `cv2.imshow` und damit die volle Version mit GUI-Unterstützung. Also wurde die headless-Variante wieder entfernt und durch die vollständige ersetzt. Zusätzlich haben wir noch die Python-Pakete nachinstalliert, die im Basis-Image fehlten: EasyOCR für die Trikotnummern-Erkennung, Pandas für die Datenverarbeitung und XGBoost für die Formationsklassifikation.

3.1.2. Modelloptimierung mittels TensorRT

Um die Inferenzgeschwindigkeit der YOLO-Modelle auf der ressourcenbeschränkten Jetson-Hardware zu erhöhen, wurden die trainierten Modelle mittels der im JetPack-Image bereits enthaltenen TensorRT-Bibliothek in das hardwareoptimierte `.engine`-Format exportiert. Der Export erfolgte mit FP16-Präzision, um Inferenzgeschwindigkeit und Genauigkeit für den Edge-Einsatz auszubalancieren. Zu berücksichtigen war hierbei, dass eine TensorRT-Engine hardwarespezifisch kompiliert wird: Der Export musste folglich direkt auf der Zielhardware (dem Jetson) selbst durchgeführt werden, da eine auf einem anderen System erstellte `.engine`-Datei nicht auf abweichender Hardware lauffähig ist.

Durch diese Optimierung konnte die Pipeline auf der Jetson-Hardware eine Verarbeitungsgeschwindigkeit von rund 10 Bildern pro Sekunde erreichen, was für eine praxistaugliche Echtzeit-Analyse am Spielfeldrand ausreicht.

3.1.3. Herausforderungen bei der Edge-Entwicklung

Auf dem Jetson zu entwickeln statt lokal am Mac war ein echter Bremsklotz. Zwei Dinge vor allem: Es gab keine vollwertige IDE direkt auf dem Gerät. Code musste also woanders geschrieben und danach erst rübergespielt werden. Umständlich, aber machbar. Das größere Problem war Docker. Jede noch so kleine Änderung an der Umgebung bedeutete einen kompletten Rebuild des Containers, und der brauchte auf der Jetson-Hardware 20 bis 30 Minuten. Bei der Planung mussten wir diese langsameren Iterationszyklen also von Anfang an mit einkalkulieren.

3.2. Hauptsteuerung und Video-Pipeline [Lenis Serna]

Die Basis des gesamten Systems bildet die Hauptsteuerungslogik, welche die Bildbereitstellung und die sequentielle bzw. parallele Ausführung der ML-Modelle orchestriert. Diese Architektur wurde in zwei zentrale Komponenten unterteilt: Die asynchrone Datenbereitstellung in `“video_pipeline.py”` und den zentralen Orchestrator bei `“main.py”`.

3.2.1. Asynchrone Bildverarbeitung

Ein direkter und synchroner Ansatz zur Videoverarbeitung mit OpenCV wäre es, die Funktion `“cv2.VideoCapture.read()”` direkt in der Hauptschleife zu nutzen. Nichtsdestotrotz stellt dies für ein rechenintensives KI-Projekt einen kritischen Bottleneck dar, da Festplattenzugriffe sowie generelle Eingabe- und Ausgabe-Operationen im Vergleich zur Verarbeitung durch den CPU oder GPU sehr langsam sind. Wenn das Hauptprogramm bei jedem Frame auf die Festplatte warten würde, käme es zu Blockaden. Diese Blockaden treiben die Inferenzzeiten der Modelle künstlich in die Höhe.

Um dieses Problem zu lösen, wurde eine dedizierte Pipeline namens "video_pipeline.py" entwickelt. Diese implementiert das sogenannte Producer-Consumer-Pattern durch den Einsatz von Multithreading. Einerseits liest ein separater Daemon-Thread als Erzeuger (Producer) die Bildern von der Festplatte und schiebt sie in einen threadsicheren Puffer namens "queue.Queue". Das Hauptprogramm agiert gleichzeitig als Verbraucher (Consumer) und ruft die Frames direkt aus dem Arbeitsspeicher ab.

Auszug aus "video_pipeline.py":

```
def update(self):
    while True:
        if self.stopped:
            return
```

Diese Funktion läuft isoliert in einem Daemon-Thread (im Hintergrund) und eine Endlosschleife hält den Thread am Leben. Dieser Prozess läuft weiter, bis die "stopped" Variable von außen gesetzt wird.

```
if not self.Q.full():
    grabbed, frame = self.stream.read()

    if not grabbed:
        self.stop()
        return

    self.Q.put(frame)
```

Solange der Puffer mit einer maximalen Größe von 128 Frames nicht voll ist, liest der Thread kontinuierlich das nächste Bild aus dem Videostream aus. Danach legt der Thread das Bild in den Puffer. Dies stellt sicher, dass dem Hauptprogramm stets ausreichend Bilder zur Verfügung stehen.

```
else:  
    time.sleep(0.01)
```

Ist der Puffer hingegen voll, bedeutet dies, dass die Festplatte schneller liest, als die KI Modelle inferieren können. In diesem Fall pausiert der Thread für 10 Millisekunden. Diese kurze Pause verhindert eine hundertprozentige Auslastung des Prozessors durch sogenanntes aktives Warten oder Busy Waiting und schont somit die Ressourcen der Hardware.

3.2.2 Evolution und Architektur des Hauptprogramms

In der initialen Phase bestand die “main.py” lediglich aus einer simplen Endlosschleife zur Videowiedergabe, währenddessen die Aufrufe für die Machine Learning Modelle dabei durch Mock-Funktionen simuliert wurden. Als die Komplexität des Projektes zunahm, wuchs die Datei zu einem vollwertigen Orchestrator heran. Dieser Orchestrator koordiniert den ganzen Datenfluss zwischen YOLO-Tracking, Homographie, Texterkennung über OCR und XGBoost.

Der zentrale Kontrollfluss wird durch eine kontinuierliche Schleife gesteuert. Diese Schleife überwacht den Status der Video Pipeline permanent.

Auszug aus "main.py":

```
while True:
    if not pipeline.more():
        time.sleep(0.01)
    if pipeline.stopped: break
    continue
```

Das Hauptprogramm fungiert hier als Verbraucher. Es prüft zunächst, ob sich Frames im Puffer befinden. Ist der Puffer leer, wartet das Programm blockierungsfrei für 10 Millisekunden. Sobald das Video komplett verarbeitet wurde, wird die Schleife ordnungsgemäß abgebrochen.

```
frame = pipeline.read()
if frame is None: break
processed_frames += 1
```

Sind Bilder vorhanden, werden diese aus dem Puffer entnommen. Ab diesem Punkt beginnt die sequentielle Verarbeitung der Modelle. Zunächst wird das Bild an die Homographie Engine übergeben. Danach folgt die YOLO Inferenz und schließlich die Weitergabe an das XGBoost Modell zur Formationserkennung.

Um die komplexe Geschäftslogik zu steuern, nutzt das Skript verschiedene Status-Varianten. Eine essenzielle Variable ist hierbei "is_play_active". Diese Variable identifiziert den Moment des Spielbeginns.

```

if not is_play_active and processed_frames > 20 and len(pooled_offense) >= 6:
    core_players = [p["pt"] for p in pooled_offense if p["role"] in ["Center",
"Quarterback"]]

    anchor_point = np.mean(core_players, axis=0) if core_players else
np.median([p["pt"] for p in pooled_offense], axis=0)

    if pre_snap_core_anchor is None:
        pre_snap_core_anchor = anchor_point
    else:
        if np.linalg.norm(anchor_point - pre_snap_core_anchor) > 15.0:
            is_play_active = True

```

Hierbei berechnet das Skript das Zentrum der Formation, basierend auf den Positionen von Center und Quarterback. Wenn sich dieses Zentrum plötzlich um mehr als 15 Einheiten bewegt, wertet das System dies als den Start des Spielzugs. Die Variable wird dann auf True gesetzt. Dadurch wird die Formationserkennung frühzeitig eingefroren, da eine Formation nach dem Beginn des Spielzugs ihre Struktur auflöst.

Schließlich, um die Navigation zwischen verschiedenen Videoclips zu ermöglichen, baut das Programm auf zwei ineinander verschachtelten Schleifen auf. Die äußere Schleife verwaltet die Liste der verfügbaren Videos. Die innere Schleife verarbeitet die einzelnen Frames des aktuell geladenen Videos, hierbei fängt der Orchestrator Benutzereingaben dynamisch ab. Dieser Überprüfungsprozess findet innerhalb der inneren Frame Schleife statt.

```

key = cv2.waitKey(1) & 0xFF
    if key == ord("q"):
        action = 'q'
        break
    elif key == ord("n"):
        action = 'n'
        break
    elif key == ord("p"):
        action = 'p'
        break

```

Ein Tastendruck speichert die gewünschte Aktion in einer Variablen und führt einen Abbruchbefehl aus. Dieser Abbruchbefehl beendet jedoch nur die innere Schleife, wodurch das aktuelle Video dadurch sofort gestoppt wird. Das Programm schließt sich an dieser Stelle aber noch nicht komplett aus. Stattdessen führt das Skript zunächst wichtige Speicherprozesse aus. Dazu gehören das Speichern der Tracking Daten in der Datenbank und das Generieren der grafischen Playcards. Nachdem diese Sicherungsprozesse vollständig abgeschlossen sind übernimmt das Ende der äußeren Schleife die finale Navigation. Das Programm entscheidet nun basierend auf der zuvor gespeicherten Aktion über den weiteren Verlauf.

```

if action == 'q':
    break
elif action == 'p':
    video_index = max(0, video_index - 1)
else:
    video_index += 1
    play_id_counter += 1

```

Wurde der Befehl zum Beenden gewählt, bricht das Programm nun auch die äußere Schleife ab und fährt das System komplett herunter. Bei dem Befehl für das vorherige Video wird der Video Index verringert. In allen anderen Fällen springt der Index zum nächsten Video. Diese saubere Trennung der Schleifen ermöglicht es dem System, große Videodatenbanken fließend nacheinander zu verarbeiten und gleichzeitig Datenverluste zu vermeiden. Ein Neustart des Skripts zwischen den einzelnen Clips war dadurch nicht mehr erforderlich.

3.2.3 Systemkonfiguration und Hardwareerkennung

Das Hauptprogramm verfügt über wichtige Konfigurationsmöglichkeiten beim Systemstart. Über Konsolenparameter kann das Verhalten der Ausführung dynamisch angepasst werden. Der Parameter für die Ausführung ohne grafische Oberfläche deaktiviert die Videowiedergabe. Dies ist besonders nützlich für die Ausführung auf einem Server oder wenn ausschließlich die Generierung der statischen Playcards am Ende der Analyse gewünscht wird. Darüber hinaus analysiert das Skript beim Start selbstständig die verfügbare Hardwareplattform. Das System erkennt automatisch, ob es sich um ein ressourcenbeschränktes Gerät handelt oder ob spezielle Beschleuniger für die Inferenz verfügbar sind.

```
def parse_args():
    parser = argparse.ArgumentParser(description="Tactical Edge
    Analysis Pipeline")

    parser.add_argument("--headless", action="store_true",
    help="Disable OpenCV GUI for headless Edge execution")

    return parser.parse_args()

def resolve_inference_device():
    if hasattr(torch.backends, "mps") and
    torch.backends.mps.is_available(): return "mps"

    if torch.cuda.is_available(): return 0
    return "cpu"
```

Die automatische Erkennung umfasst die Auswahl zwischen regulären Prozessoren und speziellen Architekturen wie Apple Silicon oder kompatiblen Nvidia Grafikkarten. Diese dynamische Zuweisung stellt sicher, dass die Modelle stets mit der bestmöglichen Beschleunigung geladen werden.

3.3. Datenspeicherung Architektur [Ayman Lahseb]

3.3.1. DatenBank:

Direkt am Anfang des Projekts wurde eine grobe SQL-Datenbank erstellt, die für die Speicherung der vom Programm generierten Daten zuständig sein sollte. Es wurde sich für eine SQL-Datenbank entschieden, da wir diese als Modul im Studium hatten und am besten damit vertraut waren. Sie wurde über drei Dateien erstellt und bedient:

a.data_base.py:

Diese Datei übernimmt das Erstellen, Strukturieren und dauerhafte Speichern aller Analyse-Ergebnisse in der SQLite-Datenbank (data-base/football_analysis.db). Die genaue Ablauf- und Funktionsweise strukturiert sich wie folgt:

- **Datei und Ordnerprüfung:** Überprüft, ob der Ordner sowie die Datenbankdatei football_analysis.db existieren, und erstellt diese, falls sie noch nicht vorhanden sind.
- **Speichern Der Daten:** Das Hauptprogramm wandelt die analysierten Daten in SQL-Befehle (INSERT INTO) um und übergibt diese an die Datenbankschnittstelle, wo sie dauerhaft in den jeweiligen Tabellen abgelegt und per Commit abgesichert werden.
- **Threading:** Ein eigener Hintergrund-Thread (DB thread) übernimmt alle Schreibvorgänge, damit das Hauptprogramm und die KI-Modelle ohne Verzögerungen (I/O-Bottlenecks) laufen.
- **Entlastung der Hardware:** Um das Jetson Nano zu entlasten, muss das Hauptprogramm nicht warten, bis die Festplatte mit dem Speichern fertig ist.

- **FIFO-Warteschlange(First in first out):** Das Hauptprogramm ruft die Funktionen der Datenbank auf und geht direkt zum nächsten Frame über. Im Hintergrund läuft der Thread, der die Aufträge nacheinander in die Datenbank schreibt. Um das System vor Speicherüberläufen zu schützen, ist diese Warteschlange auf maximal 128 Elemente begrenzt.

- **Tabellenstruktur:**
 - games: Metadaten zum Video (Datum, Dateiname) mit den Variablen:game_id (PK, Auto-Increment), date, video.
 - plays: Spielzüge und zugehörige Formationen mit den Variablen:play_id (PK, Auto-Increment), game_id (FK -> games), timestamp, formation.
 - players: Zuordnung von Tracking-IDs, Trikotnummern (OCR), Rollen und Teams mit den Variablen:id (PK), tracking_id (YOLO-Tracker-ID), jersey_number, player_role (z. B. Center, QB), team_side (Offense/Defense).
 - player_tracking: Frame-by-Frame Koordinaten (X, Y) der Spieler mit den Variablen:id (PK), tracking_id (FK -> players), play_id (FK -> plays), frame_id, x_coord, y_coord.
 - routes: Klassifizierte Laufrouten der Offense (route_id [PK], play_id [FK -> plays], tracking_id [FK -> players], route_type).

- **Batch-Tracking:** Bei hochfrequenten Daten wird in data_base.py die Funktion executemany aufgerufen (BATCH_TRACKING). Die Frame-Koordinaten aller Spieler werden dabei nicht einzeln, sondern gesammelt als Paket (Batch) in die Datenbank geschrieben. Das spart viele Festplattenzugriffe und sorgt für hohe FPS auf dem Gerät.

b.data_class.py:

Diese Datei definiert die Datenstrukturen (Data Classes) des Projekts und dient als Verbindung zwischen den KI-Pipelines und der Datenbank. Sie stellt sicher, dass alle erfassten Informationen strukturiert und im richtigen Datentyp vorliegen.

- **Aufgabe:** Sie speichert die Daten aus den Pipelines im Arbeitsspeicher genau in den Objekten, wo sie hingehören (z. B. GameData, PlayData, PlayerData, TrackingPoint, RouteData), bevor sie an die Datenbank übergeben werden.
- **Vorbereitung für die Datenbank:** Die jeweiligen Klassen spiegeln die Tabellenstruktur der Datenbank wider und stellen sicher, dass die Daten geordnet in den richtigen Tabellen landen.
- **to_tuple():** In der Klasse TrackingPoint befindet sich die Methode to_tuple(). Sie wandelt die Positionsdaten eines Spielers direkt in ein Python-Tupel um. Dies ist essenziell für das Batch-Processing (executemany) in der Datenbank, da so hunderte Tracking-Punkte extrem schnell und ohne Umwege in einem einzigen SQL-Schritt gespeichert werden können.

c. Main.py:

Was die Datenbank betrifft, speichert main.py nichts selbst, sondern ruft lediglich die Funktionen aus data_base.py auf und nutzt eine temporäre Datenhaltung für die laufenden Spielzüge (Plays):

- **Aufruf der Datenbank-Funktionen:** Die main.py führt selbst keine direkten SQL-Befehle aus, sondern steuert das Speichern über klar definierte Zeitpunkte und Schnittstellen aus data_base.py:
 - Anfang: Zu Beginn eines Videos schickt main.py ein GameData-Objekt an db.save_game(), um das Video in der Hauptdatenbank zu registrieren.
 - Pro-Frame-Zyklus: In jedem Frame werden die erfassten Positionsdaten in TrackingPoint-Objekten gebündelt und via db.save_tracking_batch() laufend an die FIFO-Warteschlange der Hauptdatenbank übergeben.
 - Clip-Ende: Erst wenn ein Video vollständig durchgelaufen ist, werden die gesammelten Spieler-Informationen (db.save_player()) sowie die erkannten Laufwege (db.save_route()) final in der Datenbank abgelegt.
 - Ende des Programms: Beim Beenden ruft main.py die Funktion db.stop() auf, um sicherzustellen, dass alle restlichen Schreibaufträge im Hintergrund abgearbeitet und die Verbindungen sauber getrennt werden.

- **Temporäre Speicherung der Spielzüge:** Zusätzlich zur Hauptdatenbank schreibt main.py nach jedem Clip die Bewegungsmatrix aller Offensivspieler über build_and_save_matrix() in eine temporäre SQLite-Datenbank (tactical_system.db) in die Tabelle offense_trajectory_matrix. Dies entkoppelt die visuelle Playcard-Generierung von der eigentlichen Hauptdatenbank.

d. Heatmap.py:

heatmap.py nutzt die Datenbank hauptsächlich für das Auslesen von Daten. Für die Erstellung der PlayCards werden die Koordinaten der Spieler über den gesamten Verlauf des Spielzugs per SQL (pd.read_sql_query) aus der temporären Tabelle offense_trajectory_matrix abgerufen. Aus diesen Positionen zeichnet das Skript die exakten Laufwege und Pfeile auf ein virtuelles Feld und speichert die Grafik als PNG-Datei im Ordner cards/ ab. Nach erfolgreicher Bildgenerierung und YOLO-Play-Klassifizierung leert das Skript die temporäre Matrix-Tabelle wieder (DELETE FROM offense_trajectory_matrix), damit die Datenbank für den nächsten Clip bereitsteht.

3.4. Spielererkennung [Ayman und Amin Lahseb, Lenis Serna]

3.4.1. Spielerlokalisierung mit YOLO

3.4.1.1. Datenerhebung und Annotation

Die Grundlage für das Training eines präzisen Objekterkennungsmodells bildet ein repräsentativer und konsistent annotierter Datensatz. Als Ausgangsmaterial dienten die Videomitschnitte aller Spiele der Saison 2024. Um eine hohe Varianz an Spielsituationen und Kameraperspektiven zu erfassen, ohne redundante Bilddaten zu erzeugen, wurden die Videos mithilfe eines Linux-Kommandozeilenbefehls via ffmpeg verarbeitet. Dabei wurden automatisiert Standbilder mit einer Frequenz von 0,5 FPS, also ein Bild alle zwei Sekunden, aus den Videostreams extrahiert.

Für die anschließende Bounding-Box-Annotation wurden die Werkzeuge Ultralytics und Roboflow evaluiert. Aufgrund der enormen Menge an zu markierenden Spielern pro Einzelbild fiel die Entscheidung auf Roboflow. Die Plattform bot entscheidende praktische Vorteile bei der Annotationsgeschwindigkeit, der Datenverwaltung sowie der nahtlosen Generierung eines konsistenten Datensatzes im YOLO-Format.

Nach dem Upload der Bilddaten wurden im Projekt fünf spezifische Klassen definiert: Center, Quarterback, Offense, Defense und Referee. Die explizite Kennzeichnung von Schiedsrichtern und Defensivspielern, obwohl der Fokus des Gesamtsystems primär auf den Angriffsspielern liegt, verfolgte einen gezielten lerntheoretischen Zweck. Durch das gezielte Annotieren von Defense und Referees wird es dem Modell deutlich leichter gemacht, visuelle Merkmale wie unterschiedliche Trikotfarben oder Positionierungsmuster klar voneinander abzugrenzen. Dies verhindert Fehldetektionen und steigert die Erkennungsgenauigkeit der relevanten Offensivakteure maßgeblich.

Zusätzlich wurde im Vorfeld evaluiert, ob auch die am Spielfeldrand stehenden Ersatzspieler und Betreuer als eigene Klasse annotiert werden sollten, um eine fehlerhafte Erfassung außerhalb des aktiven Spielgeschehens zu vermeiden. Es wurde jedoch bewusst darauf verzichtet, diese separat zu labeln. Die zugrunde liegende Hypothese war, dass das YOLO-Modell durch die räumliche Anordnung sowie die spezifischen Aufstellungsmuster (*Pre-Snap-Formationen*) selbstständig lernen würde, aktive Spieler auf dem Feld von inaktiven Personen am Spielfeldrand zu unterscheiden. Diese Annahme bestätigte sich im späteren Trainings- und Evaluierungsverlauf: Das Modell erkannte die räumlichen Muster zuverlässig und ignorierte Personen außerhalb des Spielfelds weitgehend, ohne dass ein zusätzlicher Annotationsaufwand erforderlich war.

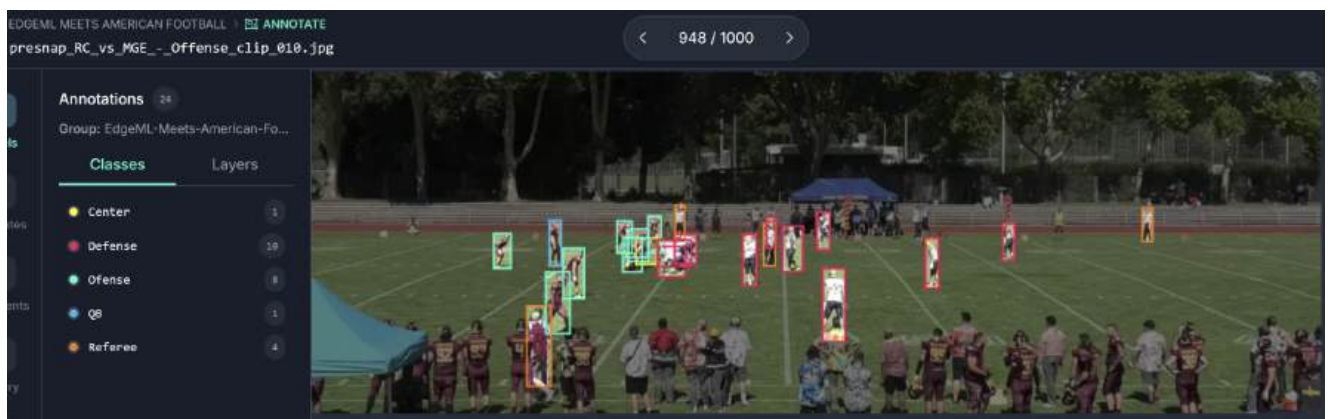


Abbildung 2: Annotationsansicht der Bilddaten in Roboflow für das Training des YOLO-Modells

3.4.1.2. Modelltraining und Optimierung

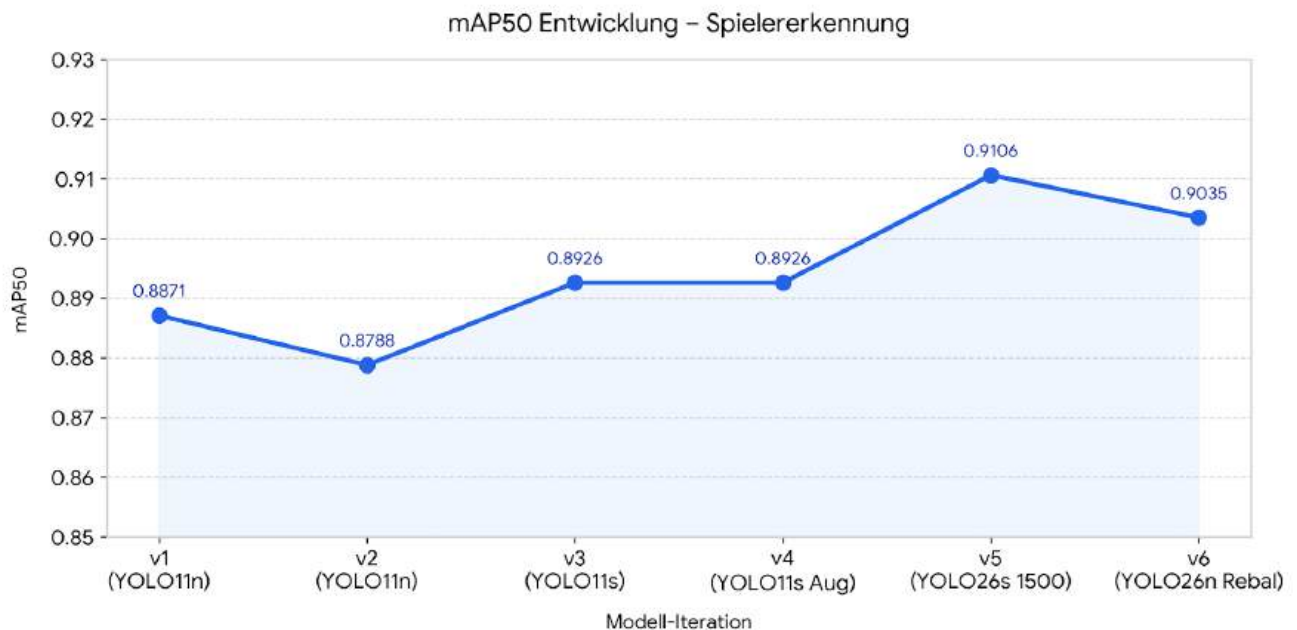


Abbildung 3 : Entwicklung der Evaluierungsmetrik (mAP@50) für die Spielererkennung über die sechs Modell-Iterationsstufen (v1 bis v6)

Das Training des Objekterkennungsmodells für die Spielererkennung erfolgte in einem iterativen Prozess über insgesamt sechs Hauptversionen (v1 bis v6), um die Erkennungsgenauigkeit und Generalisierungsfähigkeit schrittweise zu maximieren. Neben der gezielten Anpassung der Netzarchitekturen spielte insbesondere die kontinuierliche Erweiterung und Aufbereitung der Trainingsdatenbank eine entscheidende Rolle für den Gesamterfolg des Systems.

Die ersten Trainingsläufe wurden initial über die Cloud-Plattform Roboflow durchgeführt. Dies ermöglichte ein zügiges Prototyping und die schnelle Erstellung erster Baseline-Modelle (v1 und v2) auf Basis einer Datenmenge von ca. 600 annotierten Bildern. Da die kostenfreie Nutzung von Roboflow auf eine Obergrenze von 600 Bildern beschränkt ist und die Cloud-Ressourcen bezüglich der Trainingsdauer limitiert sind, wurde die gesamte Trainingsinfrastruktur ab Version v3 auf den Hochleistungsrechner der Universität migriert.

Die Nutzung lokaler Dedicated GPUs der NVIDIA RTX-Serie ermöglichte nicht nur eine erhebliche Beschleunigung der Trainingszeiten, sondern bot auch die notwendige

Flexibilität, wesentlich größere Datenmengen sowie komplexere Modellarchitekturen ohne Plattformbeschränkungen zu trainieren.

Im Zuge dieser Migration wurde der Datensatz schrittweise von den ursprünglichen 600 Bildern auf insgesamt 1.500 annotierte Einzelbilder erweitert.

In den ersten beiden Iterationen v1 und v2 kamen auf Roboflow die leichtgewichtigen YOLO11n-Modelle mit etwa 2,58 Millionen Parametern zum Einsatz. Hierbei wurde bei der Spielererkennung bereits eine starke Baseline erzielt: Version v1 startete mit einer Genauigkeit von 88,7 %, die in v2 minimal auf 87,9 % variierte. Nach dem Wechsel auf den Universitätsserver wurde in den Versionen v3 und v4 auf die leistungstärkere YOLO11s-Architektur mit ca. 9,41 Millionen Parametern skaliert. Durch den zusätzlichen Einsatz erweiterter Data-Augmentation-Techniken (v4) konnte die Robustheit gegenüber wechselnden Lichtverhältnissen und Perspektiven weiter gesteigert werden, woraufhin sich die Genauigkeit auf 89,3 % (v3 und v4) erhöhte.

Mit dem Ausbau der Datenbasis auf ca. 1.500 Einzelbilder in der fünften Iteration v5 wurde eine YOLO11s-Architektur evaluiert. Die deutlich vergrößerte Datenmenge führte zu einer erneuten Leistungssteigerung auf den Spitzenwert von 91,1 % Genauigkeit. Um schließlich dem Problem ungleich verteilter Klassen, wie etwa dem selteneren Auftreten von Quarterbacks oder Schiedsrichtern im Vergleich zu regulären Feldspielern, entgegenzuwirken, wurde für die finale Version v6 (YOLO11n Rebal) ein gezieltes Klassen-Rebalancing durchgeführt. Dies verhinderte ein Overfitting auf die dominanten Klassen und stabilisierte die Genauigkeit bei finalen 90,4 %.

Auf Einzelbildebene erreichte das Modell am Ende eine hervorragende Performance und konnte nahezu alles zuverlässig erkennen, was auch mit dem menschlichen Auge identifizierbar war. Bei der späteren Integration in die fortlaufende Video-Pipeline zeigte sich in der Praxis jedoch eine verbleibende Herausforderung:

Durch das Zusammenspiel vieler dynamischer Faktoren, wie schnellen Bewegungsschüben und frameübergreifender Verarbeitung, kam es vereinzelt noch zu visuellen Artefakten wie Ghosting. Während die isolierte Objekterkennung auf

Standbildern somit nahezu perfekt agiert, verlangt der flüssige Einsatz im Video-Stream nach weiterführenden Glättungs- und Tracking-Mechanismen.

3.4.1.3. Integration im Code und Pipeline-Ablauf

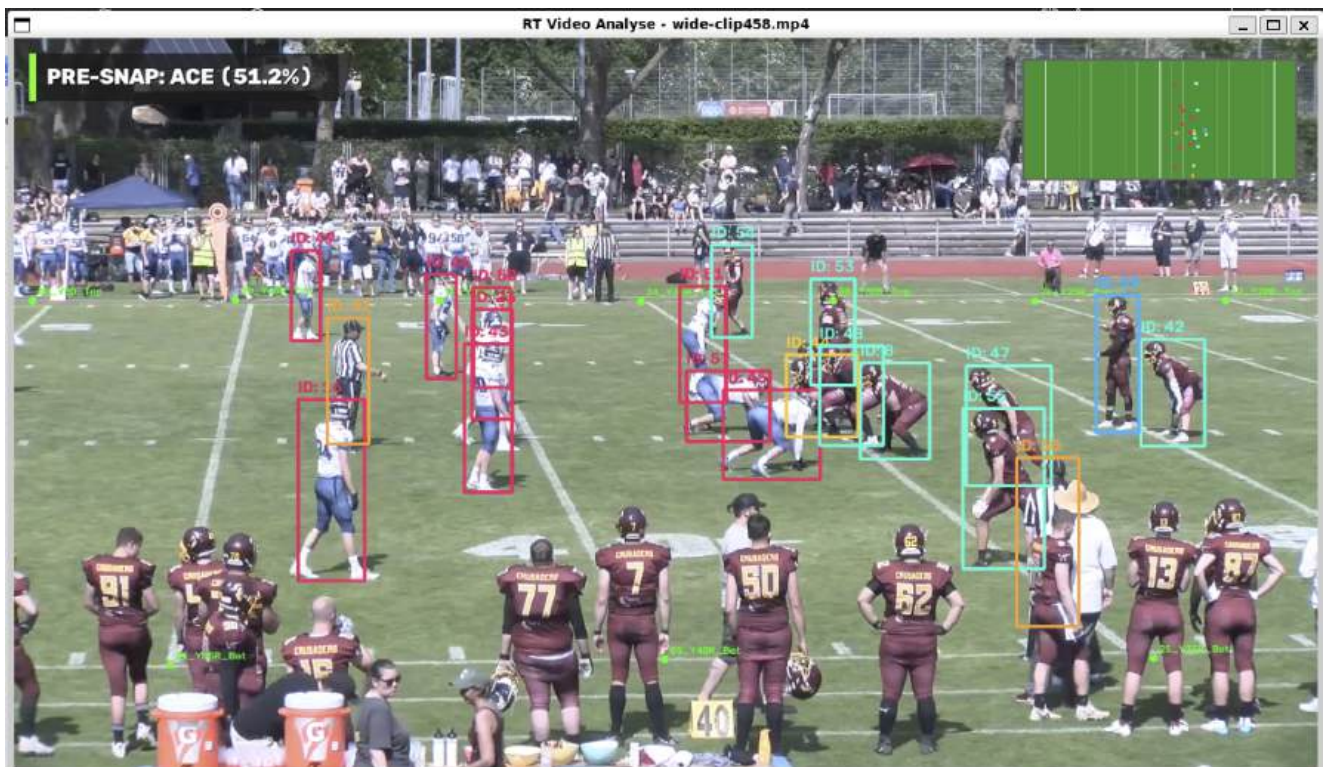


Abbildung 4: Illustration des schrittweisen Verarbeitungsprozess der entwickelten Python-Pipeline von der Videobild-Eingabe bis zur strukturierten Datenausgabe

Der Prozess beginnt mit dem kontinuierlichen Einlesen des Videostreams Frame für Frame (mittels OpenCV). Jeder einzelne Frame wird im Speicher aufbereitet und an das geladene YOLO-Modell übergeben. Innerhalb einer zentralen Inferenz-Schleife führt das Modell die Detektion durch und gibt für jedes erkannte Objekt im Frame die zugewiesene Klasse (Center, Quarterback, Offense, Defense, Referee), den zugehörigen Konfidenzwert sowie die Begrenzungsbox (Bounding Box) in Pixelkoordinaten zurück.

Zur Datenbereinigung werden unsichere Detektionen über einen Konfidenz-Schwellenwert (Confidence Threshold) direkt gefiltert. Zudem eliminiert die Non-Maximum Suppression (NMS) doppelte Bounding Boxes für dieselbe Person. Aus den verbleibenden validen Bounding Boxes berechnet der Code automatisch die exakten

Mittelpunkte (Centroids) der Spieler, welche als repräsentative Standpunkte auf dem Spielfeld dienen. Anschließend werden diese Rohdaten frame-weise aggregiert: Pro verarbeitetem Bild wird ein strukturierter Datensatz erzeugt, der die Frame-Nummer, den Zeitstempel, die Objekt-ID, die Klassenbezeichnung sowie die Mittelpunktskoordinaten (x, y) enthält. Diese Daten werden flüchtig im Hauptspeicher gesammelt und nach Abschluss der Videoanalyse in eine strukturierte CSV-Datei exportiert. Diese exportierte Datei dient als permanente Datengrundlage und Schnittstelle für die nachfolgenden Module der Pipeline, wie etwa die Perspektivtransformation auf die 2D-Taktikkarte sowie das Tracking der Spielerpfade über die Zeit.

3.4.2. Trikotnummer Erkennung und Rollenzuweisung [Lenis Serna]

Das automatisierte Ablesen von Trikotnummern amerikanischer Fußballspieler aus Videomaterial niedriger Qualität stellt eine erhebliche Herausforderung im Bereich des Computer Visions dar. Nach einer initialen Recherche und Evaluierung verschiedener Frameworks wie Tesseract und EasyOCR fiel die Entscheidung auf EasyOCR. Entscheidend hierfür waren die höhere Effizienz und Präzision dieses Modells unter nicht idealen visuellen Bedingungen, wie sie durch Bewegungsunschärfe und wechselnde Perspektiven auf einem Spielfeld typischerweise auftreten.

3.4.2.1. Erste Ansätze und Optimierung des Bildzuschnitts

Der erste Implementierungsversuch bestand in einer direkten Integration des OCR-Frameworks in die Pipeline. Hierbei wurden die vom Spielererkennungsmodell gelieferten Bounding Boxes roh an das OCR-Modell übergeben. Die Erkennungsrate erwies sich jedoch als unzureichend, da zu viel visuelles Rauschen (Hintergrund, Körperteile, verdeckende Spieler) die Textextraktion störte.

Eine tiefere Analyse ergab, dass das Modell signifikant bessere und schnellere Ergebnisse liefert, wenn das Eingabebild ausschließlich die Trikotnummer zeigt. Um den Bildzuschnitt so sauber wie möglich zu gestalten, orientierte sich das Team zunächst an dem Literaturhinweis "A General Framework for Jersey Number Recognition in Sports Video" (Koshkina et al., 2024). Der Artikel empfiehlt den Einsatz von Yolo-Pose zur

Bestimmung der Körperhaltung anhand von Schlüsselpunkten (wie Schultern und Hüften), um den Rumpf präzise zu isolieren. Obwohl dieser Ansatz erfolgreich implementiert wurde, ergaben Performancetests, dass das Pose-Modell bei dem vorliegenden Videomaterial häufig fehlschlug. In etwa 95 bis 98% der Fälle musste das System auf einen fest definierten Fallback-Zuschnitt zurückgreifen. Dieser Zuschnitt reduziert das Bild deterministisch auf den inneren, oberen Bereich der Bounding Box: 15% von oben, 10% von den Seiten und 62,5% von unten. Da der Yolo-Pose-Einsatz folglich kaum Mehrwert bot, aber wertvolle Rechenzeit beanspruchte, wurde das Modell vollständig aus der Pipeline entfernt, um die Effizienz zu steigern.

3.4.2.2. Einführung eines Lesbarkeitsmodells

Trotz des optimierten geometrischen Zuschnitts ergab sich ein neues Problem: Viele der generierten Bildausschnitte ergaben keinen Sinn (z. B. zugeschnittene Beine, Rasen oder unscharfe Rückenpartien ohne Text). Um zu verhindern, dass das OCR-Modell wertvolle Rechenleistung für unleserliche Bilder verschwendet, wurde ein Bildklassifizierungsmodell (yolo11n-cls.pt) trainiert. Dieses "Lesbarkeitsmodell" wurde auf Basis von rund 9.000 manuell annotierten Bildausschnitten aus der ursprünglichen Pipeline trainiert und erreichte eine Top-1-Accuracy von 94,82 % sowie eine Top-5-Accuracy von 100 %. Das Modell prüft nun vorab jeden Zuschnitt. Nur wenn das Bild als "legible" (lesbar) eingestuft wird, wird es zur eigentlichen Textextraktion an EasyOCR weitergeleitet.

3.4.2.3. Asynchrone Verarbeitung und Caching-Strategie

Da die OCR-Analyse trotz dieser Filterung der rechenintensivste Prozess der Architektur blieb, wurde die Trikotnummer-Erkennung in eine Klasse ("OCRAnalysisPipeline" innerhalb des Moduls "ocr_pipeline.py") ausgelagert. Diese Klasse wird zur Laufzeit als eigenständiger, asynchroner Hintergrund-Thread instanziiert und ausgeführt. Da für die abschließende Evaluierung keine Echtzeit-Erkennung in jedem einzelnen Frame zwingend erforderlich ist, verhindert diese Trennung, dass die Hauptpipeline blockiert wird. Die Kommunikation zwischen dem Hauptprogramm und dem Hintergrund-Thread erfolgt über geteilte Dictionaries, denen als shared cache dienen.

Zusätzlich wurde dem System eine intelligente Caching-Funktionalität hinzugefügt. Das Programm speichert erkannte Trikotnummern unter der jeweiligen Tracking-ID des Spielers ab. Die Hauptpipeline übergibt Spieler nur dann erneut an den OCR-Thread, wenn sie noch nicht im Cache erfasst sind oder wenn die Konfidenz der bisherigen Lesung unter 50% liegt. Wird in einem späteren Frame ein besserer Winkel mit einer höheren Konfidenz gefunden, überschreibt das System den Cache-Eintrag.

Ein verbleibendes Risiko bei dieser Methodik ist das sogenannte "ID-Switching" des YOLO-Trackers, bei dem IDs zwischen Spielern unbeabsichtigt vertauscht werden können. Dieses Tracking-Problem wurde jedoch als akzeptabler Kompromiss eingestuft. Die immense Steigerung der Prozesseffizienz und der OCR-Genauigkeit durch den Legibility-Check und das Caching überwiegt die potenziellen Fehler durch ID-Switches, welche an anderer Stelle der Pipeline algorithmisch abgemildert werden.

3.4.3. Multi-Object Tracking und Re-Identifikation (Re-ID)

Während das trainierte YOLOv6 Modell die Identifizierung und Lokalisation von Spielern auf Einzelbildebene übernimmt, erfordert die zeitliche Analyse eines Videostreams ein kontinuierliches Multi-Objekt Tracking (MOT). Das Ziel besteht darin, jedem erkannten Objekt über aufeinanderfolgende Frames hinweg eine konsistente Tracking ID zuzuweisen und zeitweilige Verdeckungen sowie Identitätsverluste mithilfe von Re-Identifikations-Mechanismen (Re-ID) auszugleichen.

Für das räumlich zeitliche Tracking der Akteure auf dem Spielfeld kommt der ByteTrack Algorithmus zum Einsatz. Im Gegensatz zu herkömmlichen Trackern verarbeitet ByteTrack nicht nur Detektionen mit hoher Konfidenz, sondern bezieht auch Bounding-Boxes mit niedrigerer Konfidenz in den Zuordnungsprozess ein. In jedem Frame werden die von YOLO ermittelten Bounding-Boxes sowie die Klassifikationen an den Tracker übergeben. ByteTrack nutzt daraufhin Kalman-Filter zur Vorhersage der zukünftigen Spielerpositionen und verknüpft diese über die Intersection over Union (IoU) mit den aktuellen Detektionen. Aus den Ergebnissen werden die Mittelpunkte der Bounding-Boxes sowie die Zuordnungen zu den Rollen wie Offense, Defense, Center, Quarterback oder Referee extrahiert und temporär vorgehalten.

Rein bewegungs- und visual-basierte Tracker neigen bei extrem schnellen Richtungswechseln, Überschneidungen mehrerer Spieler oder Kameraschwenks zu sogenannten ID-Switches. Um die Identität von Spielern auch nach einem ID-Verlust wiederherzustellen, verwendet das System eine hybride Re-ID-Strategie mittels optischer Zeichenerkennung (OCR). Erkennungsdaten von Spielern, deren Trikotnummer noch nicht im System hinterlegt ist oder deren Konfidenz unter einen Schwellenwert von 0,50 fällt, werden dynamisch an eine parallele OCR Analysis Pipeline übermittelt. Erfolgreich gelesene Trikotnummern werden anschließend in einer zentralen Cache-Struktur unter der jeweiligen Tracking ID gespeichert. Verliert der ByteTrack-Algorithmus eine ID und weist demselben Spieler nach einer Verdeckung eine neue Tracking ID zu, wird über die kontinuierliche OCR-Erfüllung die echte Trikotnummer erneut zugewiesen, wodurch die logische Identität des Spielers im Gesamtsystem gewahrt bleibt.

Die durch Tracking und Re-ID erfassten Pfade bilden die direkte Grundlage für die spätere taktische Auswertung. Hierzu werden die Fußpunkte der Bounding-Boxes über die Homographie-Matrix von Pixelkoordinaten in reale Feld-Koordinaten umgerechnet. Um Rauschen oder kurze Tracking-Aussetzer unmittelbar vor dem Spielzugbeginn zu kompensieren, werden die Positionsdaten der letzten 30 Frames in einem PreSnapTemporalPooler gepuffert und mittels Median-Filterung geglättet. Nach Abschluss eines Clips werden alle konsolidierten Tracking-Pfade schließlich zusammen mit der Re-ID-Trikotnummer in eine strukturierte Trajektorien-Matrix einer SQLite-Datenbank exportiert.

3.5. Erste Homographie durch verschiedene Ansätze [Arshad Reyah]

Ein schwieriger Teil des Projekts war die Erzeugung der Homographie, was mehr an der Erkennung des Spielfeldes an sich als an der letztendlichen Berechnung der Homographie lag. Die klassischen Methoden, die anfangs herangezogen wurden, stießen auf Grund des vorliegenden Videomaterials an ihre Grenzen. Die wiederkehrenden Herausforderungen und Probleme werden im folgenden Teil näher erläutert.

3.5.1. Hough Lines

Die erste Methode, die zur Erkennung der Yardlines auf dem Footballfeld eingesetzt wurde, basierte auf der klassischen Hough-Transformation. Dieses Verfahren gehört zu den etablierten Techniken der Bildverarbeitung, wenn es darum geht, Geraden in Bildern zu identifizieren. Die Grundidee besteht darin, dass jede mögliche Linie im Bild durch zwei Parameter beschrieben werden kann: den Abstand zur Bildmitte und den Winkel, unter dem sie verläuft. Zunächst wird das Bild in Graustufen umgewandelt und anschließend mit einem Kantenfilter wie dem Canny-Algorithmus verarbeitet. Dadurch entstehen Pixel, die potenziell Teil einer Linie sein könnten. Die Hough-Transformation sammelt diese Pixel im sogenannten Parameterraum und sucht dort nach Häufungen, die auf das Vorhandensein einer Linie hinweisen.

Logik dahinter:

```
# Canny-Kanten extrahieren
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)

# Hough-Linien erkennen
lines = cv2.HoughLinesP(
    edges,
    rho=1,
    theta=np.pi/180,
    threshold=40,
    minLineLength=120,
    maxLineGap=50
)

# Linien einzeichnen
for x1, y1, x2, y2 in lines[:, 0]:
    cv2.line(frame, (x1, y1), (x2, y2), (0, 0, 255), 2)
```

Im Kontext eines Fußballfeldes sollte dieses Verfahren die weißen Yardlines erkennen, die sich quer über das Spielfeld erstrecken. Die Idee war, dass die Yardlines im Canny-Bild als lange, zusammenhängende Kanten erscheinen und somit von der Hough-Transformation zuverlässig als horizontale Linien identifiziert werden. Nach der Erkennung sollten die zwei stärksten Linien als Referenz für die Homographie dienen, also für die Berechnung der perspektivischen Transformation des Spielfelds.

In der praktischen Anwendung zeigte sich jedoch schnell, dass die Methode auf den verwendeten Bildern nicht zuverlässig funktioniert. Die Kamera befand sich seitlich am Spielfeldrand und in einer relativ flachen Perspektive. Dadurch erscheinen die Yardlines im Bild nicht als klare, durchgehende horizontale Linien, sondern als schwach sichtbare, perspektivisch verzerrte Strukturen, die teilweise nur aus wenigen Pixeln bestehen. Hinzu kommt, dass die Yardlines häufig durch Spieler, Schiedsrichter oder Coaches verdeckt werden. In solchen Situationen erkennt der Canny-Algorithmus deutlich mehr Kanten an den Spielern als an den Linien selbst. Die Hough-Transformation interpretiert diese Kanten dann fälschlicherweise als Linien, obwohl sie nichts mit den Yardlines zu tun haben.

Ein weiteres Problem ergibt sich aus der Beschaffenheit des Rasens. Die Grasoberfläche erzeugt eine Vielzahl kleiner, unregelmäßiger Kanten, die im Canny-Bild als Rauschen erscheinen. Dadurch entstehen viele kurze Kantenfragmente, die die Hough-Transformation verwirren und dazu führen, dass sie zufällige Segmente statt echter Yardlines erkennt. Besonders kritisch ist dies bei den Amateur- oder Trainingsfeldern, auf denen die Linien oft dünn, unterbrochen oder durch Schatten kaum sichtbar sind. Die Hough-Transformation benötigt jedoch lange, zusammenhängende Kanten, um eine Linie sicher zu identifizieren. Wenn die Yardlines im Bild nur fragmentiert oder teilweise verdeckt sind, reichen die vorhandenen Kantenpunkte nicht aus, um eine stabile Linie zu erkennen.

In den fehlerhaften Ergebnissen zeigte sich dieses Problem sehr deutlich. Die Debug-Bilder enthielten erkannte Linien, die teilweise weit außerhalb des Spielfelds lagen, etwa auf der Laufbahn oder entlang heller Objekte am Spielfeldrand. In anderen Fällen wurden Linien über Spieler gezeichnet, weil deren Umrisse stärkere Kanten erzeugten als die Yardlines selbst. Die Homographie wurde zwar berechnet, aber auf Grundlage dieser falschen Linien, wodurch die gesamte Transformation unbrauchbar wurde. Die Hashmarks konnten ebenfalls nicht erkannt werden, da die Suchbereiche aufgrund der falsch erkannten Yardlines an völlig falschen Positionen lagen.



Abbildung 5: Hough Lines Ansatz

Zusammenfassend lässt sich sagen, dass die Hough-Lines-Methode für die vorliegenden Bilder ungeeignet ist. Sie funktioniert gut bei TV-Broadcast-Kameras, die das Spielfeld aus einer hohen Perspektive zeigen, sodass die Yardlines klar und durchgehend sichtbar sind. Bei einer flachen Seitenlinienkamera, wie sie in deinem Projekt verwendet wird, sind die Yardlines jedoch zu schwach, zu unterbrochen und zu oft verdeckt, sodass die Hough-Transformation keine zuverlässigen Ergebnisse liefern kann. Diese Erkenntnis führte dazu, dass eine alternative Methode entwickelt wurde, die auf der Analyse der Weiß-Pixel-Verteilung basiert und deutlich robuster gegenüber den realen Bedingungen auf dem Spielfeld ist.

3.5.2. White Pixel

Obwohl die White-Pixel-Methode im Vergleich zur Hough-Lines-Methode deutlich robuster war und insbesondere die Yardlines zuverlässig identifizieren konnte, zeigte sich bei der praktischen Anwendung ein strukturelles Problem, das letztlich dazu führte, dass die Methode nicht als finale Lösung geeignet war. Dieses Problem betrifft die Erkennung der Seitenlinien des Spielfelds. Während die diagonalen Yardlines in der Weißpixel-Analyse relativ gut sichtbar waren, gelang es der Methode nicht, die horizontalen Seitenlinien zuverlässig zu erkennen.

Logik dahinter:

```
# Weißpixel extrahieren
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (0, 0, 180), (180, 60, 255))

# Weißpixel pro Zeile summieren
hist = np.sum(mask > 0, axis=1)

# Yardline-Peaks finden
threshold = np.max(hist) * 0.4
peaks = [y for y in range(1, len(hist)-1)
         if hist[y] > threshold and hist[y] > hist[y-1] and hist[y] > hist[y+1]]

# Yardlines einzeichnen
for y in peaks:
    cv2.line(frame, (0, y), (frame.shape[1], y), (0, 0, 255), 3)
```



Abbildung 6: White Pixel Ansatz

Der Grund dafür liegt in der geometrischen Struktur des Bildes und der Kameraperspektive. Die Yardlines verlaufen quer über das Spielfeld und erscheinen in der flachen Seitenlinienperspektive als leicht diagonale, aber dennoch relativ breite und zusammenhängende weiße Bereiche. Dadurch erzeugen sie in der vertikalen Weißpixel-Projektion klare Maxima, die sich gut identifizieren lassen. Die Seitenlinien hingegen verlaufen aus Sicht der Kamera nahezu horizontal und sind durch die Perspektive stark verzerrt. Sie erstrecken sich nicht über eine einzelne Bildzeile, sondern verteilen sich über viele Zeilen hinweg. In der Weißpixel-Projektion entsteht dadurch kein klarer Peak, sondern eine diffuse Verteilung, die von der Methode nicht als Linie interpretiert werden kann.

Hinzu kommt, dass die Seitenlinien in vielen Bildern teilweise verdeckt sind durch Spieler, Coaches, Ausrüstung oder Schatten. Während die Yardlines aufgrund ihrer Position im Spielfeld oft zumindest teilweise sichtbar bleiben, sind die Seitenlinien besonders anfällig für Verdeckungen. Die White-Pixel-Methode ist jedoch darauf angewiesen, dass die weißen Linien im Bild klar erkennbar sind.

Sobald die Seitenlinien unterbrochen oder abgeschwächt sind, bricht die Erkennung vollständig zusammen. Dies führt dazu, dass die Methode zwar die Yardlines zuverlässig identifizieren kann, aber keine vollständige geometrische Beschreibung des Spielfelds liefert.

Diese Einschränkung ist besonders kritisch für die Berechnung der Homographie. Eine stabile Homographie benötigt mehrere eindeutig identifizierbare Referenzlinien oder -punkte, die die geometrische Struktur des Spielfelds definieren. Wenn nur die Yardlines erkannt werden, fehlt ein wesentlicher Teil der Information, um die Perspektive korrekt zu rekonstruieren. Die Hashmarks können zwar als zusätzliche Punkte dienen, sind aber aufgrund ihrer geringen Größe und der flachen Kameraperspektive ebenfalls nicht immer zuverlässig erkennbar. Die fehlende Erkennung der Seitenlinien führt daher zu einer instabilen oder unvollständigen Homographie, die für die weitere Verarbeitung des Spielfelds nicht ausreichend präzise ist.

Diese Erkenntnis machte deutlich, dass eine rein pixelbasierte Methode über Weißpixel nicht ausreicht, um die geometrische Struktur des Spielfelds zuverlässig zu erfassen. Der nächste Lösungsansatz bestand darin, die Perspektive des Bildes selbst auszunutzen und nicht die Linien direkt zu erkennen, sondern die Fluchtpunkte, die durch perspektivisch parallele Linien entstehen. Die Vanishing-Point-Methode analysiert die Richtung der Linien im Bild und bestimmt die Punkte, an denen sich diese Richtungen im Unendlichen schneiden. Dadurch lässt sich die Orientierung des Spielfelds rekonstruieren, selbst wenn einzelne Linien nur teilweise sichtbar sind oder gar nicht erkannt werden.

Die White-Pixel-Methode war somit ein wichtiger Zwischenschritt, der die Erkennung der Yardlines erheblich verbesserte, aber aufgrund der fehlenden Erkennung der Seitenlinien nicht als finale Lösung dienen konnte.

3.5.3. Vanishing Points

Nachdem sowohl die Hough-Lines-Methode als auch die White-Pixel-Analyse keine vollständige und stabile Erkennung der Spielfeldlinien ermöglicht hatten, wurde als nächster Ansatz die Vanishing-Points-Methode untersucht. Diese Methode basiert nicht auf der direkten Erkennung einzelner Linien, sondern auf der geometrischen Struktur des Bildes selbst. Sie nutzt die Tatsache, dass parallele Linien im dreidimensionalen Raum wie die Yardlines, Hashmarks oder Seitenlinien eines Fußballfeldes in einer perspektivischen Projektion auf einen gemeinsamen Fluchtpunkt zulaufen. Durch die Bestimmung dieser Fluchtpunkte lässt sich de facto die Orientierung des Spielfelds rekonstruieren, ohne dass die Linien vollständig sichtbar sein müssen. Theoretisch eignet sich dieser Ansatz besonders gut für Bilder, in denen Linien teilweise verdeckt oder nur fragmentarisch sichtbar sind.

Die Funktionsweise der Vanishing-Points-Methode lässt sich wie folgt beschreiben: Zunächst werden im Bild Kanten extrahiert, die potenziell zu den relevanten Linien gehören könnten. Anschließend werden diese Kanten in Richtung ihrer Ausrichtung gruppiert. Linien, die im realen Spielfeld parallel verlaufen, beispielsweise die Yardlines sollten im Bild ähnliche Richtungen aufweisen. Durch das Verlängern dieser Linien bis zu ihrem Schnittpunkt entsteht ein Fluchtpunkt, der die perspektivische Richtung dieser Linien repräsentiert. Für ein Fußballfeld existieren typischerweise mehrere solcher Fluchtpunkte: einer für die Yardlines, einer für die Seitenlinien und ein weiterer für die Hashmark-Linien. Wenn diese Fluchtpunkte zuverlässig bestimmt werden können, lässt sich daraus eine vollständige Homographie ableiten, die das Spielfeld korrekt in die Draufsicht transformiert.

Logik dahinter:

```
# Kanten extrahieren
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)

# Liniensegmente finden
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 50, minLineLength=80)

directions = []
points = []

# Linienrichtung sammeln
for x1, y1, x2, y2 in lines[:, 0]:
    direction = np.array([x2 - x1, y2 - y1])
    direction = direction / np.linalg.norm(direction)
    directions.append(direction)
    points.append(np.array([x1, y1]))

# Fluchtpunkt berechnen (Least Squares)
A = []
b = []
for p, d in zip(points, directions):
    A.append([d[1], -d[0]])
    b.append(d[1]*p[0] - d[0]*p[1])

A = np.array(A)
b = np.array(b)

vanishing_point = np.linalg.lstsq(A, b, rcond=None)[0]
```

In der Theorie ist die Vanishing-Points-Methode äußerst robust, da sie nicht darauf angewiesen ist, dass die Linien vollständig sichtbar sind. Es genügt, dass genügend Bildstrukturen vorhanden sind, die die Richtung der Linien andeuten. In der Praxis zeigte sich jedoch, dass die Methode auf den vorliegenden Bildern nicht zuverlässig funktioniert. Der entscheidende Grund dafür liegt in der extrem flachen Kameraperspektive, die als Grundlage in diesem Projekt verwendet wurde. Die Kamera befindet sich seitlich am Spielfeldrand und nur wenige Meter über dem Boden. Dadurch erscheinen die Linien im Bild nicht als klar erkennbare, perspektivisch zulaufende Strukturen, sondern vielmehr als nahezu horizontale oder leicht diagonale Fragmente, die kaum eine eindeutige Richtung besitzen.

Ein weiteres Problem besteht darin, dass die relevanten Linien, insbesondere die Seitenlinien, häufig durch Spieler, Coaches, Ausrüstung oder Schatten verdeckt sind. Die Vanishing-Points-Methode ist darauf angewiesen, dass genügend Linienfragmente vorhanden sind, die eine gemeinsame Richtung erkennen lassen.

Wenn diese Fragmente fehlen oder durch andere Bildstrukturen überlagert werden, entstehen keine stabilen Fluchtpunkte. In dem vorhandenen Videomaterial dominieren die Spieler visuell das Geschehen. Ihre Körper, Helme, Trikots und Schatten erzeugen starke Kanten, die von der Methode fälschlicherweise als Linien interpretiert werden. Diese Kanten besitzen jedoch keine gemeinsame Richtung und führen zu einer Vielzahl inkonsistenter Schnittpunkte, die keinen sinnvollen Fluchtpunkt ergeben.

Hinzu kommt, dass die Yardlines in der flachen Perspektive nicht als lange, durchgehende Linien erscheinen, sondern als kurze, perspektivisch verzerrte Segmente, die sich kaum über mehrere Meter erstrecken. Die Vanishing-Points-Methode benötigt jedoch Linien, die über eine gewisse Länge sichtbar sind, um ihre Richtung zuverlässig bestimmen zu können. Wenn die Linien nur aus wenigen Pixeln bestehen oder durch Spieler unterbrochen sind, ist die Berechnung der Fluchtpunkte mathematisch instabil. Die Seitenlinien, die theoretisch besonders gut geeignet wären, sind in vielen Bildern entweder vollständig verdeckt oder aufgrund der Perspektive so stark verzerrt, dass sie nicht als konsistente Richtung erkannt werden können.

Die fehlerhaften Ergebnisse der Methode zeigen dieses Problem deutlich. Die berechneten Fluchtpunkte lagen oft weit außerhalb des Bildes oder verteilten sich über große Bereiche, ohne eine klare Struktur zu erkennen zu geben. In einigen Fällen entstanden Fluchtpunkte, die durch zufällige Kantenfragmente erzeugt wurden und keinerlei Bezug zum Spielfeld hatten. Die daraus berechnete Homographie war entsprechend unbrauchbar: Sie verzerrte das Spielfeld stark, führte zu inkonsistenten Transformationen oder erzeugte überhaupt keine gültige Projektion. Selbst in Bildern, in denen die Yardlines teilweise sichtbar waren, reichten die vorhandenen Informationen nicht aus, um stabile Fluchtpunkte zu bestimmen.

Zusammenfassend lässt sich sagen, dass die Vanishing-Points-Methode zwar theoretisch ideal für die Rekonstruktion der Spielfeldgeometrie ist, in der praktischen Anwendung jedoch an den spezifischen Bedingungen der vorliegenden Bilder scheiterte. Die Kombination aus flacher Kameraperspektive, verdeckten Linien, starken

Spielerstrukturen und perspektivisch verzerrten Markierungen führte dazu, dass keine konsistenten Fluchtpunkte berechnet werden konnten. Aus diesem Grund war die Methode nicht geeignet, eine stabile Homographie zu erzeugen und die Suche nach einem funktionierenden Prototyp setzte sich fort.

3.5.4. Ausblick zur funktionsfähigen Homographie Berechnung/Erkennung

Im Laufe der Zeit wurde schnell klar, dass diese Ansätze nicht hinreichende bzw. sinnvolle Ergebnisse erzielen werden. Die Ansätze wurden letztlich verworfen, da sich eine bessere Idee durch weiterführende Recherche ergeben hat, welche im folgenden Kapitel von Grund auf erläutert werden. Hier kurz angerissen mit dem ersten Ansatz der Keypoint Annotierung und dem Trainieren eines yolov-8n-pose Modells zur akkuraten performanten Homographie Berechnung und Erkennung. Mit diesem Modell wurde allerdings keine ausreichende Genauigkeit ($<10\%$) erzielt, weshalb mehrere bis schlussendlich alle Linien annotiert werden mussten, um das beste Ergebnis zu erzielen.

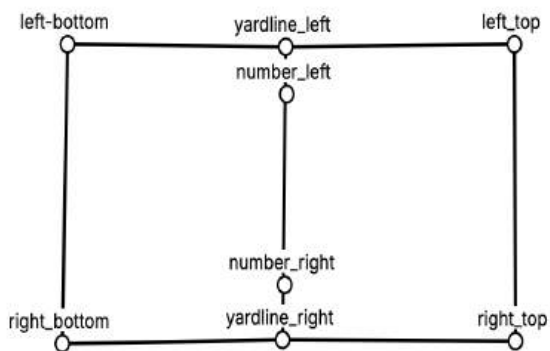


Abbildung 7: Erster Ansatz mit Mittellinie und Numbers

3.5.5. Erkenntnis

Die drei Ansätze haben gezeigt, dass rein pixelbasierte Methoden für die real-gegebene Kameraperspektive ungeeignet sind und "mühsames" händisches Annotieren robuster und zuverlässiger funktioniert.

3.6. Homographie Engine [Ayman und Amin Lahseb, Lenis Serna]

Ein wichtiger Teil des Projekts war die Berechnung der Homographie. Sie sorgt dafür, dass das schräge Kamerabild mathematisch in eine flache, zweidimensionale Ansicht des Spielfelds umgerechnet wird.

Als Grundlage dafür erkennt ein KI-Modell feste Orientierungspunkte (Keypoints) auf dem Feld. Das Erstellen der Trainingsdaten und das Festlegen dieser Punkte wurden schrittweise in drei Trainingsphasen umgesetzt.

3.6.1. Erstes Training (1. Iteration)

Die erste Trainingsphase gestaltete sich wie folgt:

- **Datensatz:** Für den ersten Versuch wurden ca. 250 Einzelbilder (Pre-Snap-Szenen) mithilfe eines Skripts aus verschiedenen Videoclips extrahiert und auf Roboflow hochgeladen.
- **Skelett:** Das erste Skelett war sehr einfach aufgebaut und bestand aus nur 10 Keypoints (den Yard-Linien von 30 links bis 30 rechts, jeweils mit einem oberen und unteren Punkt). Verdeckte Punkte (z. B. durch Spieler) wurden als „occluded“ markiert (Abbildung.8).
- **Training:** Auf dieser Basis wurde ein yolov8n-pose-Modell (v1) über 100 Epochen trainiert.
- **Ergebnis & Problem:** Das erste Modell (v1) erreichte eine Bounding-Box-Genauigkeit von 97,2 % Box mAP@50, womit das Spielfeld als Gesamtobjekt gut erkannt wurde. Die Genauigkeit der Schlüsselpunkte lag jedoch bei 0,0 % Pose mAP@50.
- **Der Grund:** Stand die Kamera weit links oder rechts auf dem Feld, befanden sich kaum oder gar keine der 10 definierten Punkte im Bildausschnitt. Dem Modell fehlten dadurch die notwendigen Orientierungspunkte, um das Feldskelett zu erkennen.

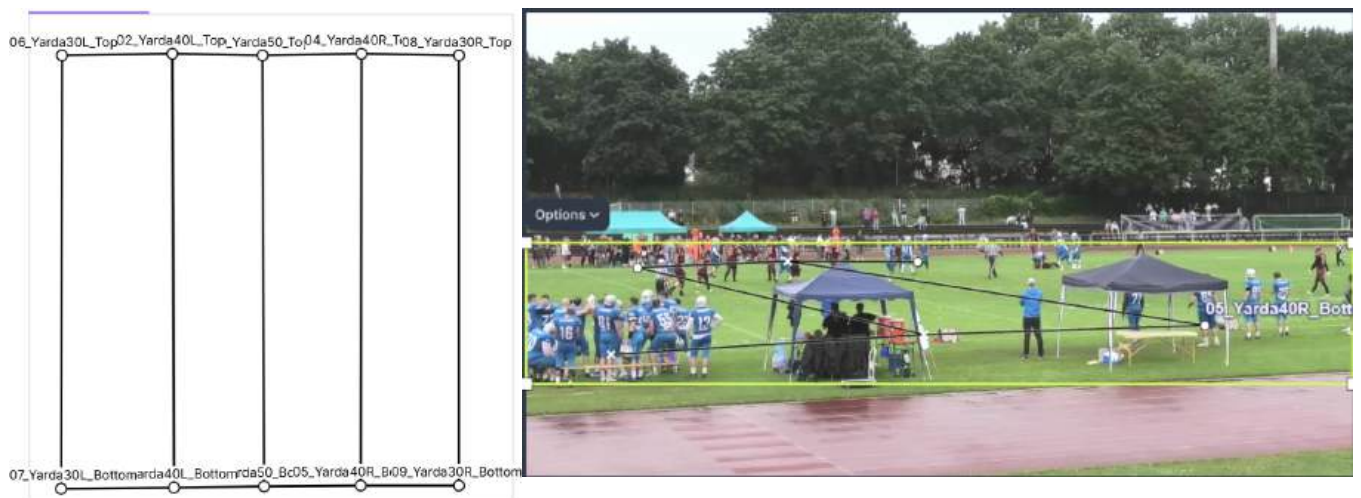


Abbildung 8: Links ist die zweite Version des Spielfeld-Skeletts dargestellt, zusammen mit einem entsprechend annotierten Beispielbild (rechts)

3.6.2. Zweites Training (2. Iteration)

Die zweite Trainingsphase gestaltete sich wie folgt:

- **Datensatz-Erweiterung:** Zusätzlich zu den ursprünglichen 250 Bildern wurden 700 weitere Frames gezielt mithilfe eines Python-Skripts extrahiert. Das Skript nutzte das bisherige v1-Modell, um in jedem Video-Clip automatisch gezielt Fehlfälle (sog. Hard Examples) mit fehlerhafter Homographie zu identifizieren. Jeder Video-Clip wurde zeitlich in vier Abschnitte (Quartale) unterteilt. Pro Quartal extrahierte das Skript genau einen Frame, bei dem mindestens eines der folgenden vier Qualitätskriterien für eine fehlerhafte Homographie erfüllt war:
 - Zu wenige Keypoints : Es wurden weniger als 6 Keypoints im Bild erkannt.
 - Geringer Inlier-Konsens (RANSAC-Filter): Ein RANSAC-Algorithmus filterte geometrische Ausreißer. Ein Frame wurde ausgewählt, wenn mehr als 25 % der erkannten Punkte mathematisch/logisch nicht zusammenpassten.

- Ungültige Matrix-Ausgabe: Alle erkannten Keypoints lagen auf einer einzigen geraden Linie, wodurch keine zweidimensionale Fläche aufgespannt werden konnte.
- Horizont-Verletzung: Das Modell sagte Punkte außerhalb der eigentlichen Spielfeldbegrenzung voraus.

- **Annotation:** Die bereits vorhandenen Frames wurden überarbeitet (re-annotiert) und gemeinsam mit den neu hinzugefügten Frames an das erweiterte Skelett angepasst.
- **Skelett:** Das Skelett wurde um die 10er- und 20er-Yard-Linien (Rechts und Links) ergänzt (insgesamt 18 Keypoints) (Abbildung 9).
- **Training:** Für die Trainingsläufe kamen die Architekturen yolo11n-pose und yolo26s-pose über jeweils 100 Epochen zum Einsatz.
- **Ergebnis & Erkenntnis:** Die Überarbeitung lieferte den Durchbruch: Während die Objekt-Genauigkeit stabil bei 99,3 % bis 99,5 % Box mAP@50 verblieb, stieg die Punkt-Genauigkeit schlagartig von 0,0 % auf bis zu 95,4 % Pose mAP@50 (v9) bzw. 96,7 % (v10).
- **Verbleibendes Problem:** Trotz der sehr guten Gesamtergebnisse traten in speziellen Szenen weiterhin Homographie-Fehler auf. Bei der Fehleranalyse zeigte sich, dass in engen Kamera-Ausschnitten oft nur unbeschriftete 5er-Yard-Linien sichtbar waren. Da diese nicht im 18-Punkte-Skelett enthalten waren, fehlten dem Modell in diesen Nahaufnahmen eindeutige Orientierungspunkte.

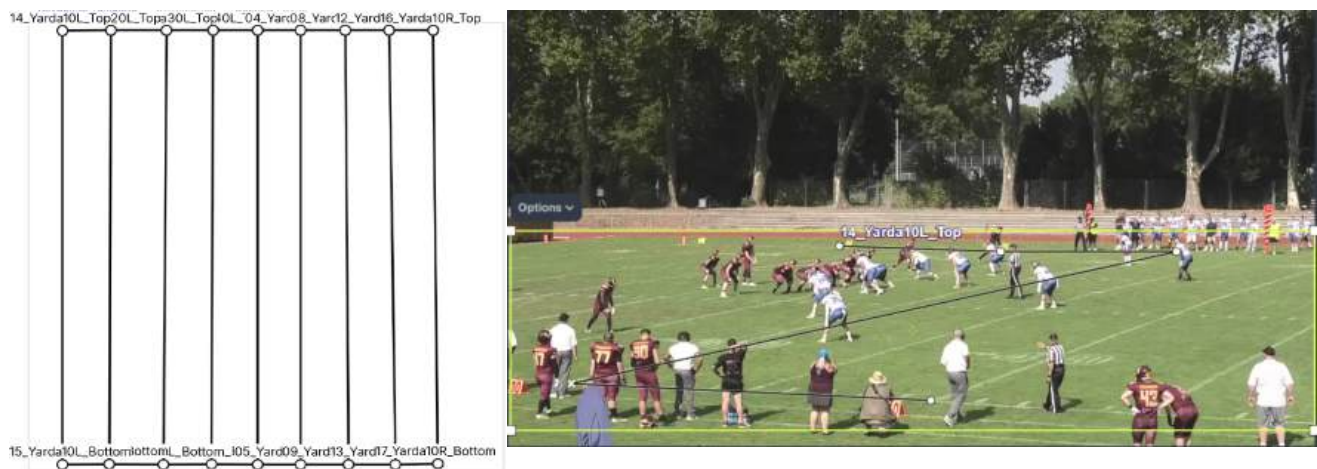


Abbildung 9: Links ist die zweite Version des Spielfeld-Skeletts dargestellt, zusammen mit einem entsprechend annotierten Beispielbild (rechts)

3.6.3. Drittes Training (3. Iteration)

Die dritte Trainingsphase gestaltete sich wie folgt:

- **Vollständiges Skelett:** Das Skelett wurde komplett überarbeitet und um alle 5er- und 10er-Yard-Linien, die Seitenlinien sowie die Endzonen-Begrenzungen erweitert (insgesamt 46 Keypoints). Dadurch mussten alle 1.000 Bilder an das neue, hochdetaillierte Raster angepasst werden.
- **Annotation:** Um den manuellen Annotationsaufwand zu reduzieren, wurden zunächst 500 Bilder manuell an das 46-Punkte-Skelett angepasst. Mit diesen Daten wurde ein Zwischenmodell trainiert, das für das automatische Pre-Labeling der verbleibenden 500 Bilder eingesetzt wurde. Im Anschluss mussten die automatisch generierten Keypoints nur noch manuell überprüft und nachkorrigiert werden (Abbildung 10).
- **Training :** Für das finale Training kam das Modell yolo26n-pose (Konfiguration tests/v3) über 500 Epochen zum Einsatz.
- **Ergebnis:** Das finale Modell erreichte mit einer Punktgenauigkeit von 96,6 % Pose mAP@50 sowie einer Bounding-Box-Erkennung von 99,4 % Box mAP@50 (models/feldererkennung/tests/v3) das angestrebte Ziel. Es bietet eine exzellente

Balance zwischen hoher Präzision und schneller Verarbeitungsgeschwindigkeit für den mobilen Einsatz auf Edge-Geräten.

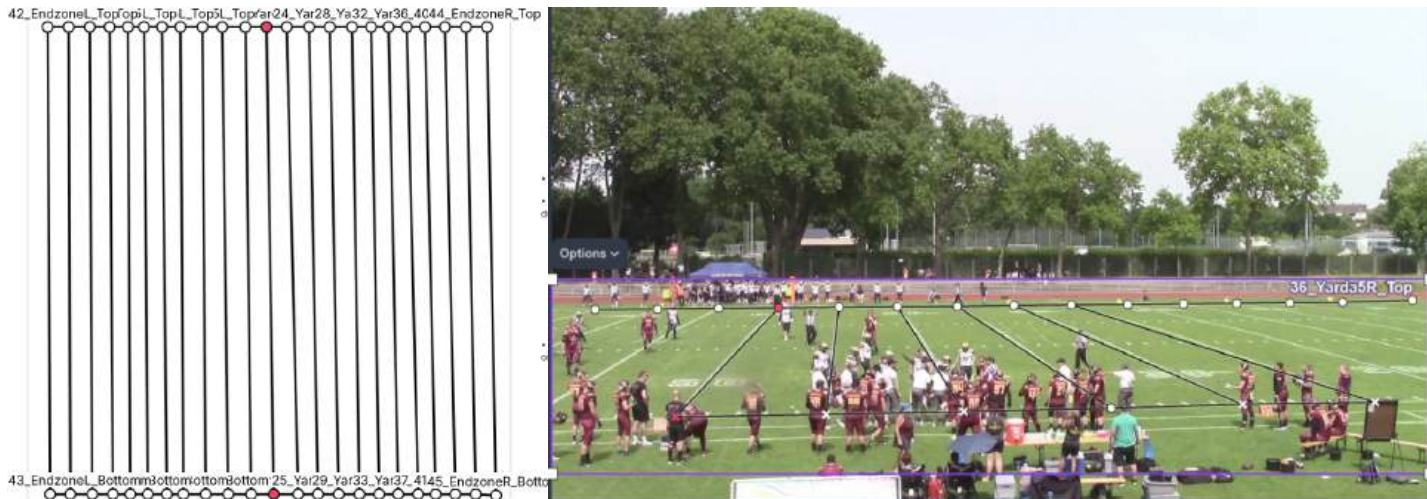


Abbildung 10: Links ist das finale Spielfeld-Skelett dargestellt, zusammen mit einem entsprechend annotierten Beispielbild (rechts)

3.6.4. Integration und Optimierung der Homographie-Engine

Die Felderkennung und die darauf basierende Homographie-Berechnung wurden als eigenständige Modulkasse HomographyEngine in der Datei homography_engine.py strukturiert und nahtlos in die Haupt-Pipeline (main.py) integriert. Die gesamte Architektur der Koordinatentransformation baut auf dieser Klasse auf. Um eine optimale Ausführungsgeschwindigkeit auf unterschiedlichen Zielsystemen zu gewährleisten, erkennt der Code beim Start dynamisch die verfügbare Hardware-Beschleunigung.

Er unterstützt dabei NVIDIA GPUs (cuda), Apple Silicon (mps) sowie einen automatischen Fallback auf die CPU.

- **Geometrische Plausibilitätsprüfung** : Zur Erhöhung der Erkennungsgenauigkeit beinhaltet die Pipeline eine geometrische Plausibilitätsprüfung anhand einer dynamisch berechneten Median-Linie („Y-Äquator“). Diese prüft, ob als oben definierte Keypoints auch tatsächlich in der oberen Spielfeldhälfte liegen. Punkte, die auf der falschen Seite erkannt werden, identifiziert das System als Ausreißer,

markiert sie für das visuelle Debugging rot und schließt sie von der Matrixberechnung aus.

- **Mathematische Stabilisierung und Fallback:** Um das unvermeidbare Kamera-Wackeln bei Live-Übertragungen auszugleichen, wird die Homographie-Matrix über eine zeitliche Dämpfung (Exponential Moving Average, EMA mit $\alpha = 0,85$) geglättet. Kann in einem einzelnen Frame kein valides Spielfeld erkannt werden (z. B. durch Verdeckungen), greift das System automatisch auf die zuletzt berechnete, gültige Transformationsmatrix zurück, um Kontinuität im Tracking zu garantieren.
- **Performance-Optimierung durch Frame-Skipping und Visuelles Caching:** Zur Entlastung der GPU muss die KI-Inferenz nicht zwingend auf jedem Einzelbild laufen. Stattdessen wird die Homographie-Inferenz typischerweise auf jeden zweiten oder dritten Frame begrenzt. In den dazwischenliegenden Frames wird die GPU-Belastung für die Felderkennung um 100% reduziert. Damit das visuelle Overlay für den Benutzer nicht flackert, speichert die Engine die gezeichneten Punkte in einem visuellen Cache und zeichnet sie auf den übersprungenen Frames neu nach. Dies ermöglicht hohe Bildraten (FPS) und ein flüssiges Overlay.
- **Standardisierung des 2D-Spielfeldes und RANSAC-Schätzung:** Das finale Ergebnis der Homographie ist die Projektion auf ein standardisiertes 2D-Taktikfeld mit einer Auflösung von 1200×533 Pixeln. Hierbei entspricht der Maßstab präzise $10 \text{ Pixel} = 1 \text{ Yard}$. Für die eigentliche Berechnung der Transformationsmatrix H kommt der RANSAC-Algorithmus (Random Sample Consensus) zum Einsatz. Analog zur Datensatz-Generierung filtert dieser verbliebene mathematische Ausreißer zuverlässig heraus und sorgt für eine stabile Transformation der Spielerpositionen.

3.7. Offensive Formationerkennung [Lenis Serna]

3.7.1. Erste Versuche: Bildbasierte Klassifikation mit YOLO

Das Ziel am Anfang war es, die offensive Aufstellung (Formation) direkt aus den Bildern der Pre-Snap-Phase zu erkennen. Der erste Versuch bestand darin, ein Bildklassifizierungsmodell mit YOLO zu trainieren. Dieser Ansatz lieferte jedoch sehr schlechte Ergebnisse und übertraf nie eine Genauigkeit von 40 bis 45 %. Die Hauptgründe dafür waren die hohe Anzahl an verschiedenen Formationsklassen und ein starkes Ungleichgewicht (Class Imbalance) im Dataset. Zudem waren die Bilder oft komplex und von niedriger Qualität, was es dem Modell extrem schwer machte, mit verdeckten Spieler umzugehen (Occlusions) und echte Formationsstrukturen zu lernen.

3.7.2. Das Fehlen der räumlichen Perspektive

Zu diesem Zeitpunkt fehlte dem Projekt noch eine funktionierende Perspektivkorrektur. Die Entwicklung der Homographie-Engine verzögerte sich stark, da das ursprünglich dafür verantwortliche Teammitglied diese Aufgabe nicht erfüllte. Ohne diese Korrektur versuchte das Team zunächst, ein Random-Forest-Modell auf tabellarischen Daten zu trainieren, die direkt aus den rohen Pixelkoordinaten der Spieler bestanden. Auch dieser Ansatz scheiterte ($< 45\%$), da die rohen Pixelwerte ohne Perspektivkorrektur keinen echten geometrischen Sinn ergaben, zum Beispiel waren Abstände im Vordergrund und Hintergrund nicht vergleichbar oder 50 Pixeln entsprachen nicht immer den gleichen Abstand, sondern kam das auf die Perspektive des Videos an. Z

udem konnte Random Forest schlecht mit fehlenden Werten (Null-Werten) umgehen, wenn Spieler nicht erkannt wurden.

In einem weiteren Versuch, die Homographie zu umgehen, entwickelte das Team ein Python-Skript, das synthetische Bilder generierte, wobei die Koordinaten der Spieler als einfache weiße Kreise auf einem schwarzen Hintergrund dargestellt wurden. Die Hoffnung war, dass YOLO diese stark vereinfachten Muster besser lernen könnte. Da die Koordinaten jedoch weiterhin verzerrt waren und das Klassenungleichgewicht bestehen blieb, brachte auch dieser Ansatz nicht den gewünschten Erfolg.

3.7.3. Der Durchbruch: Homographie und der Wechsel zu XGBoost

Erst nachdem die Homographie-Engine von anderen Teammitgliedern erfolgreich entwickelt und stabilisiert wurde (siehe hierzu ausführlich Kapitel 3.4. Homographie Engine), änderte sich die Situation grundlegend. Durch die Projektion der Pixel auf eine 2D-Ebene aus der Vogelperspektive ergaben die Distanzen zwischen den Spielern und dem Center endlich geometrischen Sinn.

Mit diesen korrekten Daten entschied sich das Team gegen Random Forest und für den Einsatz von XGBoost. Recherchen zeigten, dass XGBoost besonders leicht ist und sich hervorragend für Edge-Geräte eignet. Der entscheidende Vorteil war jedoch die native Fähigkeit von XGBoost, mit fehlenden Werten (NaN) in Tabellen umzugehen, was für unvollständiges Player-Tracking ideal war.

3.7.4. Datenbankstruktur und gezielte Datenaugmentation

Für das Training wurde das Skript `formation_architect.py` entwickelt. Dieses Skript greift direkt auf die erstellte SQLite-Datenbank ("`tactical_system.db`") zu und extrahiert die stabilsten Pre-Snap-Phasen, um sie in einer bereinigten Tabelle ("`formations_curated`") zu speichern. Um das XGBoost-Modell widerstandsfähig gegen Tracking-Ausfälle zu machen, wurde der Datensatz stark augmentiert. Neben leichtem räumlichen Rauschen wurde gezielt eine Simulation von verdeckten Spielern (Occlusions) programmiert. Hierbei wurden absichtlich Koordinaten von 1 bis 4 Spielern gelöscht und durch NaN (Not a Number) ersetzt:

Synthetische Simulation von fehlenden YOLO-Erkennungen (Occlusions) durch NaN

Auszug aus: formation_architect.py

```
num_dropped = np.random.randint(1, 5)
dropped_indices = np.random.choice(range(2, 11), size=num_dropped, replace=False)
aug_coords[dropped_indices] = np.nan
```

Durch diese Technik lernte das Modell, Formationen auch dann korrekt zu klassifizieren, wenn der Tracker in der realen Anwendung Spieler verliert.

3.7.5. Zeitliches Pooling zur Stabilisierung in der Inferenz

Um das Modell in der Live-Inferenz weiter zu stabilisieren, wurde im Hauptskript (main.py) ein Mechanismus implementiert, der als PreSnapTemporalPooler bezeichnet wird. Dieser aggregiert die erkannten Spielerpositionen über ein rollierendes Zeitfenster von 30 Frames.

Auszug aus: main.py

```
class PreSnapTemporalPooler:
    """Aggregiert Spielererkennungen über ein rollierendes Zeitfenster,
    um einzelne Frame-Ausfälle zu überstehen."""
    def __init__(self, max_frames=30):
        self.buffer = deque(maxlen=max_frames)

    def update(self, players_list):
        self.buffer.append(players_list)
```

Diese Logik sammelt die Positionen der Spieler. Fällt eine Erkennung für wenige Frames aus, wird die Position aus der Historie (Median der letzten 30 Frames) rekonstruiert.

3.7.6. Ergebnis

Durch die Kombination der Homographie-Engine, der gezielten NaN-Augmentation in der Datenbank und des zeitlichen Poolings konnte eine enorme Verbesserung erzielt werden. Das XGBoost-Modell erreichte bei der Evaluierung auf dem Testdatensatz eine finale Validierungsgenauigkeit (Accuracy) von 96.45%. Dies beweist, dass strukturierte Koordinaten in Kombination mit XGBoost für dieses Problem weitaus effizienter sind als reine Bildklassifikationsansätze.

3.8. Routen-Nachzeichnung[Amin Lahseb]

3.8.1. Speicherung und Pflege der Laufkoordinaten

Die Rekonstruktion der Laufwege (Routen) basiert auf der schrittweisen Transformation zweidimensionaler Bounding-Boxen aus dem Videobild in ein normalisiertes zweidimensionales Spielfeldkoordinatensystem. Der softwareseitige Ablauf gliedert sich in vier zentrale Teilprozesse: (1) Bestimmung des Bodenkontaktpunkts, (2) perspektivische Transformation mittels Homographie, (3) Persistierung und Aufbereitung der Trajektorien sowie (4) Signalglättung und Interpolation.

1. Bestimmung des Bodenkontaktpunkts

Aus den vom Objekterkennungsmodell YOLO bereitgestellten Bounding-Boxen der detektierten Spieler wird die untere Mitte der Bounding-Box als repräsentativer Referenzpunkt für die Fußposition bestimmt.

Zunächst werden die Eckpunkte der Bounding-Box extrahiert:

```
x_min, y_min, x_max, y_max = s["box"]
```

Anschließend werden die Fußkoordinaten im Kamerabild berechnet:

```
feet_x = (x_min + x_max) / 2.0  
feet_y = float(y_max)
```

Der Punkt `(feet\_x, feet\_y)` dient als Ausgangspunkt für die nachfolgende Transformation in das Spielfeldkoordinatensystem.

2. Perspektivische Transformation mittels Homographie

Zur Umrechnung der Bildkoordinaten “(feet_x, feet_y)” in reale Spielfeldkoordinaten wird die vom Modul `HomographyEngine` berechnete Transformationsmatrix “current_H” verwendet. Die perspektivische Projektion erfolgt mithilfe der OpenCV-Funktion “cv2.perspectiveTransform()”. Zunächst wird der Punkt im für OpenCV erforderlichen Format vorbereitet:

```
feet_coord = np.array([[[feet_x, feet_y]]], dtype=np.float32)
```

Anschließend erfolgt die Transformation auf das Spielfeld:

```
mapped_point = cv2.perspectiveTransform(feet_coord, current_H)
field_x = float(mapped_point[0][0][0])
field_y = float(mapped_point[0][0][1])
```

Die resultierenden Koordinaten `(field\_x, field\_y)` repräsentieren die Position des Spielers im zweidimensionalen Spielfeldkoordinatensystem.

3. Persistierung und Aufbereitung der Trajektorien

Die berechneten Spielfeldkoordinaten werden im weiteren Systemverlauf in zwei Schritten verarbeitet.

3.1. Batch-Persistierung

Während der Videoanalyse wird für jeden verarbeiteten Frame ein `TrackingPoint`-Objekt erzeugt und im Rahmen einer Batch-Verarbeitung in der relationalen Datenbank `tactical\_system.db` gespeichert.

```
tracking_batch.append(TrackingPoint(
    tracking_id=player_id,
    play_id=current_play_id,
    frame_id=processed_frames,
    x_coord=final_x,
    y_coord=final_y
))
```

Dadurch wird die vollständige zeitliche Folge aller Spielerpositionen persistiert.

3.2. Generierung der Trajektorienmatrix

Nach Abschluss der Videoanalyse konsolidiert die Funktion `build\_and\_save\_matrix()` die einzelnen Trackingpunkte in der Tabelle `offense\_trajectory\_matrix`. Jede Zeile der Matrix repräsentiert einen identifizierten Offensivspieler mit seiner Tracking-ID, der zugewiesenen Rolle sowie der erkannten Trikotnummer. Die Spalten enthalten die Koordinaten für die jeweiligen Frames.

```
row_entry = {
    "YOLO_Tracking_ID": player_id,
    "Role": frames_data.get("role", "Offense"),
    "Jersey_Number": jersey if jersey is not None else "-"
}
for f in range(1, total_frames + 1):
    row_entry[f"Frame_{f}"] = frames_data.get(f, "-")
```

Die Trajektorienmatrix bildet die Grundlage für die spätere Routenanalyse und Visualisierung.

4. Signalglättung und Interpolation

Um Messrauschen sowie Fehldetektionen im zeitlichen Verlauf der Bewegungsdaten zu reduzieren, werden fehlende Koordinaten zunächst linear interpoliert. Anschließend werden die X- und Y-Koordinaten mit einem eindimensionalen Gauß-Filter ("gaussian_filter1d") geglättet.

```
smoothed_x = gaussian_filter1d(interpolated_x, sigma=2)
smoothed_y = gaussian_filter1d(interpolated_y, sigma=2)
```

Die Glättung reduziert kurzfristige Positionsschwankungen und führt zu kontinuierlichen, realistischeren Laufwegen. Die so erzeugten Trajektorien werden abschließend für die Visualisierung auf den generierten Playcards verwendet.

3.8.2. Automatische Generierung von PlayCards

Hier ist das gesamte Kapitel 3.6.2 in einer **kompakten, flüssigen Fließtext-Fassung für deine Projektarbeit**, komplett ohne Code-Snippets:

Die automatische Erstellung von zweidimensionalen Taktikkarten („PlayCards“) übersetzt die verarbeiteten und geglätteten Bewegungskordinaten in ein visuelles Format zur Spielzuganalyse. Das Modul `generate_numbered_yard_playcard` liest hierzu die Trajektorien-Matrix aus der SQLite-Datenbank aus und rendert die Grafik mittels Matplotlib. Dieser Prozess untergliedert sich in vier wesentliche Schritte:

1. **Daten Extraktion und Vorbereitung:** Die gespeicherten Positionsdaten und Trikotnummern werden aus der Datenbank ausgelesen. Lückenhafte Werte werden linear interpoliert und anschließend mittels 1D-Gauß-Filter geglättet, um kontinuierliche Routenverläufe ohne Rauschen zu gewährleisten.

2. **Zeichnen des Spielfelds und der Line of Scrimmage:** Auf einem maßstabsgetreu gerenderten Spielfeld werden Yard-Linien, Hash-Marks und Zahlenmarkierungen platziert. Zudem wird die Line of Scrimmage anhand der Pre-Snap-Position des Centers berechnet und als vertikale, gestrichelte Linie in einer Kontrastfarbe über das gesamte Feld gezeichnet, um die Anspielinie eindeutig zu markieren.
3. **Visualisierung von Aufstellung und Routen:** Die Offensivspieler werden an ihren Startpositionen vor dem Snap als Symbole gezeichnet (mit spezifischen Kennzeichnungen für Schlüsselpositionen wie Center und Quarterback). Von diesen Startpunkten aus werden die geglätteten Trajektorien als durchgezogene Linien visualisiert. Am Endpunkt jeder Route wird automatisch eine Pfeilspitze in Laufrichtung platziert und die zugehörige Trikotnummer der Spieler ergänzt.
4. **Metadaten-Einbindung und Export:** Die Karte erhält eine Kopfzeile mit der vom XGBoost-Modell erkannten Offensiv-Formation inklusive der zugehörigen Konfidenzwerte. Die fertige PlayCard wird schließlich als hochauflösende Bilddatei im Ausgabeverzeichnis abgespeichert.

3.8.3. beispiel einer generierten Playcard



Abbildung 11: Beispielhafte PlayCard-Visualisierung für den Videoclip wide-clip151

Die Grafik verdeutlicht die visuelle Aufbereitung der aggregierten und geglätteten Bewegungskoodinaten:

- Feld-Grid und Line of Scrimmage (LOS): Das Spielfeld ist maßstabsgetreu mit Yard-Linien und Hash-Marks dargestellt. Die vertikale, blau hervorgehobene Linie markiert die Line of Scrimmage (LOS) knapp hinter der 50-Yard-Linie und bildet den Ausgangspunkt des Spielzugs.
- Metadaten-Overlays: In der linken oberen Ecke befinden sich die Klassifikationsergebnisse des ML-Modells. Der Spielzug wurde mit einer Konfidenz von 82,0 % als ACE-Formation und mit 64,0 % als FADE-Play identifiziert.
- Spieleraufstellung und Routenverläufe: Die Startpositionen (Pre-Snap) der Offensivspieler sind durch Kreise markiert, die anhand der Tracking-ID sowie der jeweiligen Rolle (z. B. Cent für Center, Quar für Quarterback, Atta für Attacker/Skill-Player) gelabelt sind. Wo erkannt, ist zusätzlich die Trikotnummer angegeben (z. B. # 20).

- Trajektorien und Pfeilspitzen: Die geglätteten Laufwege der Spieler werden als farbige Vektorlinien dargestellt. Pfeilspitzen am Ende jeder Linie zeigen die finale Bewegungsrichtung und das Routenende der jeweiligen Spieler an.

3.8.4. PlayCards-Benutzeroberfläche (GUI)

Zur interaktiven Visualisierung und einfachen Überprüfung der generierten PlayCards wurde eine grafische Benutzeroberfläche (High-Res Cards Viewer) auf PySide6-/PyQt-Basis implementiert. Abbildung 12 zeigt den Aufbau und die Struktur der Anwendung.



Abbildung 12: Benutzeroberfläche des PlayCards Viewers zur Inspektion der generierten PlayCards

Beschreibung der Anwendungselemente:

1. Vorschau- und Zoom-Bereich (CARD PREVIEW): Das zentrale Hauptfenster stellt die ausgewählte PlayCard dar. Über einen Doppelklick auf die Grafik kann direkt in das Bild hineingezoomt werden, um z. B. eng beieinanderliegende Laufrouen an der Line of Scrimmage im Detail zu analysieren. Alternativ lässt sich das Bild über eine Vollbildansicht öffnen.
2. Aktions-Buttons (Navigation & Vollbild): Über den Button „Up“ lässt sich im Ordnerpfad eine Ebene höher navigieren. Der Button „View Full Size“ öffnet die aktuell ausgewählte PlayCard in einer hochauflösenden Vollbildansicht.
3. Auswahlliste der PlayCards: Auf der linken Seite werden alle im Zielordner vorhandenen PlayCard-Grafiken (z. B. wide-clip151_playcard.png) aufgeführt. Durch Klicken auf einen beliebigen Eintrag wird die entsprechende Karte sofort im Vorschaufenster (1) geladen.
4. Speicherdatenpfad: Die obere Pfadzeile zeigt den genauen Verzeichnispfad an, in dem die von der Pipeline generierten PlayCards abgelegt und ausgelesen werden (z. B. /home/amin/infprojekt/gruppe1/cards).

3.9. Play Recognition [Ayman Lahseb]

3.9.1. Modell Trainieren:

Das Modelltraining wurde über zwei grundlegende Ansätze evaluiert:

a. XGBoost (Extreme Gradient Boosting)

1. Versuch:

Im ersten Versuch wurden die extrahierten x- und y-Koordinaten der Spieler aus den ersten 270 Videoclips der Saison 2024 in einer SQLite-Datenbank strukturiert abgespeichert.

Über die eindeutigen Clip-IDs wurden diese Bildkoordinaten anschließend mit den Spielzug-Metadaten aus der Datei Breakdown.xlsx zusammengeführt (Feat-Merging). Das erste XGBoost-Modell wurde zunächst auf eine Multi-Klassen-Klassifikation trainiert, um sämtliche im Datensatz enthaltenen Spielzugtypen (z. B. Rush, Complete Pass, Incomplete Pass, Sack) direkt zu erkennen.

- **Ergebnisse:** Über verschiedene Versuche und Iterationen hinweg (inklusive Variationen des Hyperparameter-Tunings und der Feature-Skalierung) verblieb die Modellgenauigkeit (Accuracy) im niedrigen Bereich von 20% bis 35%.
- **Vereinfachung auf binäre Klassifikation:** Um die Komplexität des Zielraums zu reduzieren, wurde das Modell im Anschluss darauf beschränkt, lediglich zwischen zwei Hauptkategorien zu unterscheiden: Passspiel (Pass) vs. Laufspiel (Run). Durch diese Umstellung auf eine binäre Klassifikation stieg die Genauigkeit des Modells auf etwa 50% bis 54% an (was nahezu der Baseline eines zufälligen Ratens entspricht).

- **Grund:** Die Ursachen für die geringe Genauigkeit im ersten Versuch lagen vor allem in den methodischen Limitationen der Datenerfassung. Zum einen stammten die extrahierten Koordinaten ausschließlich aus einzelnen Standbildern (Snapshots) zum Zeitpunkt des Ballanspiels. Ohne zeitversetzte Bewegungstrajektorien und Geschwindigkeitsvektoren nach dem Snap fehlte dem Modell der notwendige dynamische Kontext, um eine klare Unterscheidung zwischen Pass- und Laufspielzügen zu erlernen. Zum anderen führten Verdeckungen (Occlusions) innerhalb der dichten Spieler-Cluster an der Line of Scrimmage regelmäßig zu Detektionsausfällen der YOLO-Pipeline. Die daraus resultierenden fehlenden Werte (Missing Values / NaNs) erzeugten unvollständige Feature-Matrizen, was die Modellstabilität zusätzlich beeinträchtigte. Schließlich verschärfte die Aufteilung der lediglich 270 Clips auf eine Vielzahl spezifischer Spielzugtypen das Problem der Datensparsamkeit (Class Imbalance), da für einzelne Unterklassen schlicht zu wenige Trainingsbeispiele zur Verfügung standen, um verlässliche Muster zu erlernen.

2. Versuch:

Um die im ersten Versuch festgestellten Probleme zu beheben, wurden anstelle von 270 nun alle 610 Clips der Saison 2024 herangezogen, um eine neue Datenbank mit den Spielerkoordinaten aufzubauen. Diese erweiterten Daten wurden erneut mithilfe der Breakdown-Datei in das Modell integriert. Um dem Modell mehr Kontext zu bieten, wurden zusätzlich zu den Koordinaten folgende neue Features eingespeist:

- Feldposition (Yard Line)
- Formation (z. B. Ace, Deuce)
- Spielrichtung
- Distanz (Distance)

- **Ergebnisse:** Trotz der deutlichen Vergrößerung der Datenbank und der Anreicherung durch zusätzliche Kontextinformationen verbesserte sich die Performanz nicht signifikant. Die Genauigkeit des Modells lag weiterhin bei enttäuschenden 20 % bis 30 %.

b. YOLO

Nachdem die Versuche mit den XGBoost-Modellen nicht geklappt hatten, wurde ein anderer Ansatz mithilfe von YOLO ausprobiert. Es entstand die Idee, die vom Programm generierten Play Cards für das Training zu nutzen, um das Modell auf visuelle Muster anstatt auf reine Daten zu trainieren.

Ein Skript ließ mithilfe des Play-Cards-Mechanismus alle 610 Clips aus dem Jahr 2024 durchlaufen und erstellte dafür Play Cards, die den Ablauf des Spiels zeigten. Nach dem Analysieren und Zuordnen der Daten mithilfe der „Breakdown“-Datei ließ sich feststellen, dass die verschiedenen Plays nicht gleichmäßig verteilt waren – es gab beispielsweise Plays mit rund 30 Clips, während andere nur 1 oder 2 hatten.

Deshalb wurde ein Skript erstellt, das für jedes Play die unbrauchbaren Play Cards herausfilterte und neue Play Cards auf Basis der bestehenden generierte. Dafür wurden die Play Cards ein- und ausgezoomt, nach rechts, links, oben und unten verschoben oder in einem bestimmten Winkel geneigt, bis die Anzahl der Play Cards 5.000 überschritt.

- **Ergebnisse:** Mit den über 5.000 Play Cards wurde ein YOLOv11-Modell für 500 Epochen angesetzt. Das Training wurde jedoch nach ca. 5 Stunden und 105 Epochen abgebrochen. Die Ergebnisse lagen bei einer Top-5-Accuracy von 44,2 % und einer Top-1-Accuracy von 13,5 %.

- **Schlussfolgerung:** Nach dem Training mit verschiedenen Modellen und Ansätzen lässt sich die geringe Genauigkeit vor allem durch die schwankende und instabile Qualität der Videoclips erklären. Zusammen mit den inkonsistenten Kamerabewegungen und -winkeln führt dies dazu, dass die visuellen Daten nicht ausreichend übereinstimmen. Dies verwirrt das Modell und verhindert, dass es klare, wiedererkennbare Muster erlernen kann.

3.9.2. Anwendung:

Die Integration des YOLO-Modells wurde hauptsächlich in den Dateien main.py und heatmap.py umgesetzt. Das Modell wird erst nach dem Ende eines Clips und dem Generieren der Play Card aufgerufen.

Die generierte Play Card wird zunächst temporär gespeichert, damit das Klassifikationsmodell das Bild laden, den Spielzug-Typ (Play Type) bestimmen und diesen zusammen mit dem Konfidenzwert (in %) zurückgeben kann. Diese Ergebnisse werden zusammen mit der Formation oben links auf der Play Card in zwei verschiedenen Textfeldern eingefügt. Die fertige Play Card wird anschließend im Ordner cards gespeichert, während die temporäre Arbeitsdatei sofort nach der Auswertung wieder gelöscht wird.

Zur Fehlerbehandlung wird beim Aufrufen des Modells geprüft, ob die Datei best.pt (das Modell) tatsächlich im Ordner existiert. Sollte die Datei nicht vorhanden sein oder ein Fehler während der Ausführung auftreten, fängt ein try-except-Block diesen Fehler ab. Anstatt dass das Programm abstürzt oder abbricht, wird dadurch ein entsprechender Fehlertext generiert und als Hinweis auf der Play Card eingeblendet.

Darüber hinaus ist das Programm hardwareoptimiert. Da KI-Modelle sehr rechenintensiv sind und eine entsprechende Last auf der GPU erzeugen was zu Abstürzen oder hohen Latenzen führen kann, wird der Inferenz-Pfad für das gesamte Modell GPU-abhängig optimiert (wie in Abschnitt 3.2 erwähnt).

3.10. Webanwendung und lokaler Webserver [Lenis Serna]

Um den praktischen Mehrwert des Systems für die Trainer am Spielfeldrand zu maximieren, wurde in der finalen Entwicklungsphase eine responsiv gestaltete Webanwendung implementiert. Obwohl dieses Feature ursprünglich nicht zwingend in der Projektbeschreibung des Forschungsprojektes lag, wurde es als essenzielle Schnittstelle hinzugefügt, um die komplexe Datenverarbeitung des Programms nutzerfreundlicher zu visualisieren, damit die Trainer einen Mehrwert darauf haben könnten, ohne wirklich Kenntnisse in dem IT-Bereich haben zu müssen.

Alle zugehörigen Dateien für das Backend und Frontend wurden im Wurzelverzeichnis des Projektes unter dem Ordner “project-web-hub” gekapselt.

3.10.1. Architektur und Technologiestack

Die Architektur der Webanwendung folgt einem modernen, entkoppelten Ansatz basiert auf das Client-Server Muster:

- Backend (API & Video-Streaming): Als serverseitiges Framework wurde FastAPI gewählt, da es asynchrone Prozesse nativ unterstützt und hochperformant ist. Das Backend wird über den ASGI-Server Uvicorn bereitgestellt. Um die KI-Inferenz vom Webserver zu entkoppeln, wurde eine modifizierte Version des Hauptskripts (main.py) unter dem Namen “main-webhub-integration.py” erstellt. Diese Version implementiert eine “AnalysisBridge-Klasse”, die die Modelle in einem separaten Thread ausführt und die berechneten Videobilder als kontinuierlichen MJPEG-Stream (“/video_feed”) an das Frontend sendet.

- Frontend (UI & Telemetrie): Die Benutzeroberfläche wurde mit React.js und dem Build-Tool Vite entwickelt. Das Frontend kommuniziert sich mit dem Backend über REST-Endpoints für Steuerungsbefehle (Start, Pause, Videoauswahl) und über eine persistente WebSocket-Verbindung ("/ws/telemetry"), um Echtzeit-Telemetriedaten (wie Inferenz-Zeiten und erkannte Spieler) im Millisekundentakt zu empfangen.

3.10.2. Offline-Deployment als lokaler Hotspot

Eine der wichtigsten Anforderungen im Amateur American Football ist die Unabhängigkeit von einer stabilen Internetverbindung auf dem Spielfeld. Aus diesem Grund wurde das Nvidia Jetson-Board so konfiguriert, dass es ein eigenes lokales WLAN-Netzwerk (Hotspot) aufspannt.

Während der Entwicklungs- und Testphase wurde der Dienst direkt auf dem Jetson über zwei separate Terminal-Instanzen gestartet:

1. Frontend: Gestartet via folgenden Terminal-Befehl, um den Zugriff im gesamten lokalen Subnetz zu erlauben.

```
pnpm run dev -- --host
```

2. Backend: Gestartet via folgenden Terminal-Befehl.

```
python3 -m uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Durch diese Offline-Subnetz-Architektur können sich Trainer mit ihren eigenen Smartphones, Tablets oder Laptops in das WLAN des Jetsons einwählen und über den Browser auf das Dashboard zugreifen, ohne dass mobile Daten oder eine externe Internetverbindung benötigt werden. Dies ermöglicht eine echte Entkopplung Cloud-Dienste und schafft eine viel freundlicher, effizienter und zugänglicher Lösung.

3.10.3. Benutzeroberfläche und Echtzeit-Dashboard

Die Benutzeroberfläche wurde vollständig responsiv gestaltet, sodass sie sich an die Bildschirme mobiler Geräte anpasst. Sie teilt sich in zwei Kernbereiche auf:

1. Video Player und Navigation: Der Nutzer kann aus einer Liste der lokal auf dem Jetson gespeicherten Videos wählen. Der Videoplayer zeigt den kontinuierlichen Stream an, auf dem die Bounding-Boxen der Spieler und das 2D-Taktikfeld (Minimap) als Overlay in Echtzeit gerendert werden.

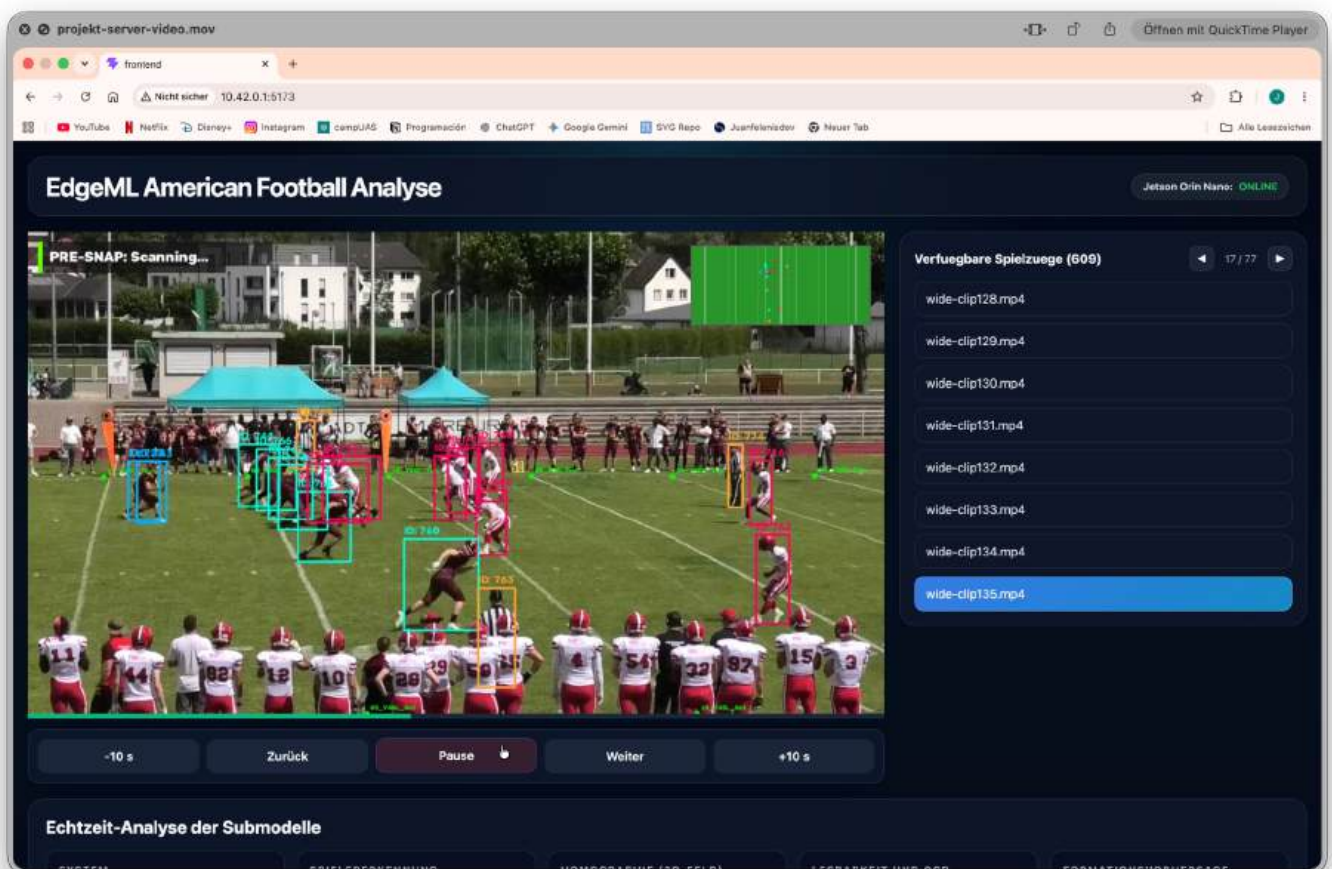


Abbildung 13: Hauptansicht der Webanwendung mit Video-Stream, 2D-Taktikfeld, Video-Steuerung und Videos-Verzeichnis Pagination

2. Telemetrie- und Metriken-Dashboard: Unterhalb des Videoplayers befindet sich ein Dashboard, das die WebSocket-Daten visualisiert. Es liefert den Trainern und Entwicklern tiefe Einblicke in die Hardware-Performance (FPS, Energieverbrauch in Watt) sowie die Inferenzzeiten der einzelnen Submodelle (YOLO, Homographie, XGBoost).

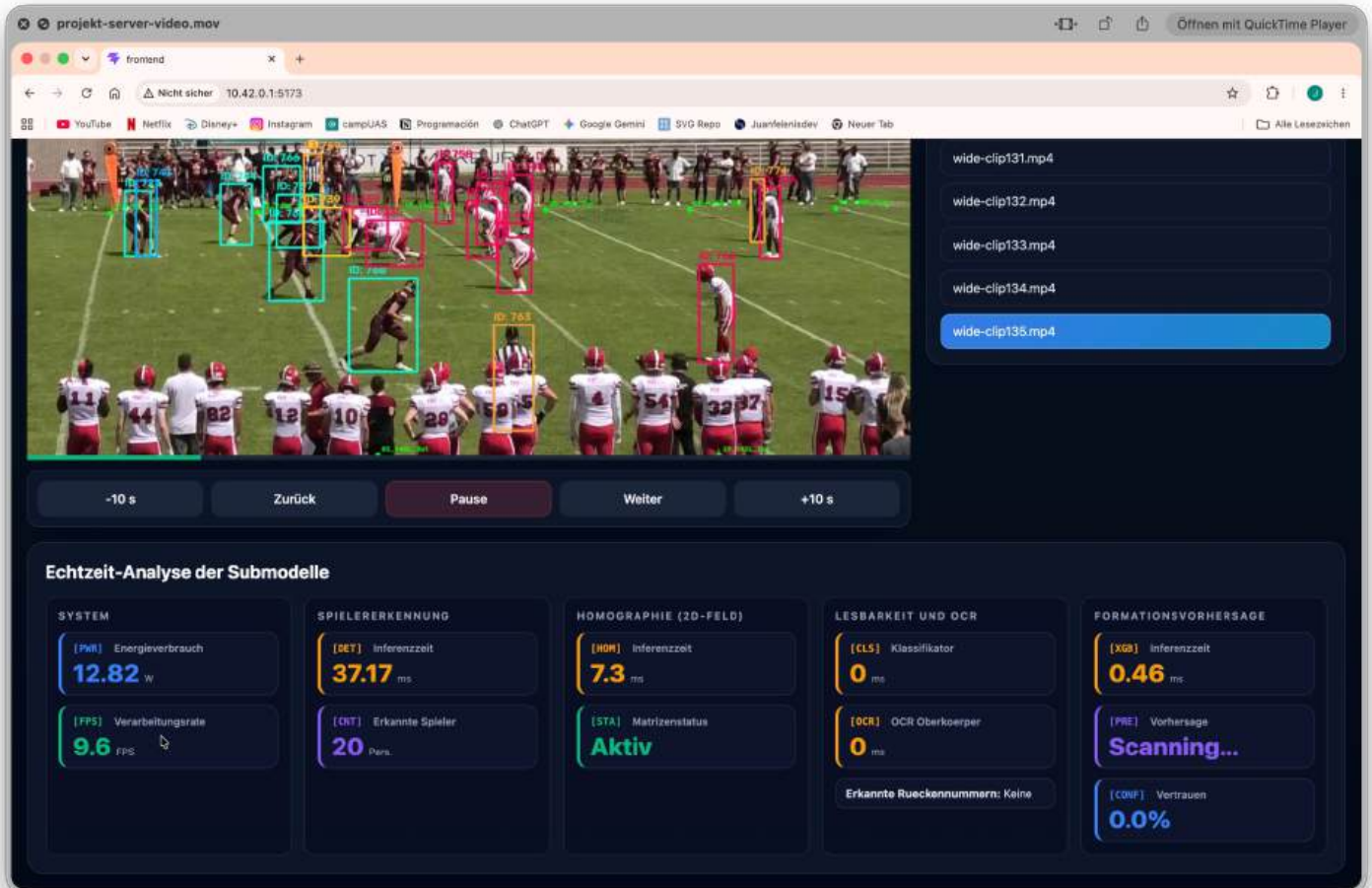


Abbildung 14: Echtzeit-Analyse Dashboard zur Überwachung der Modell-Latenzen und Hardware-Ressourcen

3.10.4. Limitationen und zukünftige Entwicklungen

Da das Web-Hub und das dazugehörige Design erst spät im Projektverlauf als Zusatz-Feature ("Added Value") entwickelt wurden, unterliegt das aktuelle System gewissen zeitbedingten Einschränkungen, die im Rahmen dieses studentischen Projekts nicht vollständig iteriert werden konnten:

- **Fehlende Feature-Parität:** Einige Funktionen, die bei der lokalen Ausführung des Skripts `main.py` bereits fehlerfrei funktionieren, konnten noch nicht in das Web-Dashboard integriert werden. Dazu gehört die finale Generierung der statischen "Playcards" sowie die vollständige Anzeige der Rückennummern (OCR-Pipeline), die im asynchronen Web-Kontext noch Instabilitäten aufweist.
- **Vision für den Produktionsbetrieb (Plug & Play):** Das langfristige Ziel für dieses System ist eine vollständige Automatisierung. Zukünftig soll das Jetson-Board so konfiguriert werden, dass es beim Einschalten automatisch den Hotspot aktiviert und die kompilierten React-Dateien sowie das Backend als Hintergrunddienst (Daemon) startet (Plug & Play).
- **Datenquelle (Live vs. Pre-Recorded):** Aktuell analysiert das System lokal gespeicherte Videos. Da die Performance-Metriken (FPS und Latenz) den Echtzeit-Anforderungen bereits genügen, wäre im nächsten Schritt der direkte Anschluss einer Webcam zur Live-Analyse des Spielfelds problemlos möglich. Alternativ muss ein einfacher Workflow (z.B. via Bluetooth) etabliert werden, damit Trainer neue Videoaufnahmen unkompliziert von ihren Smartphones auf das Jetson-Gerät übertragen können.

4. Integration

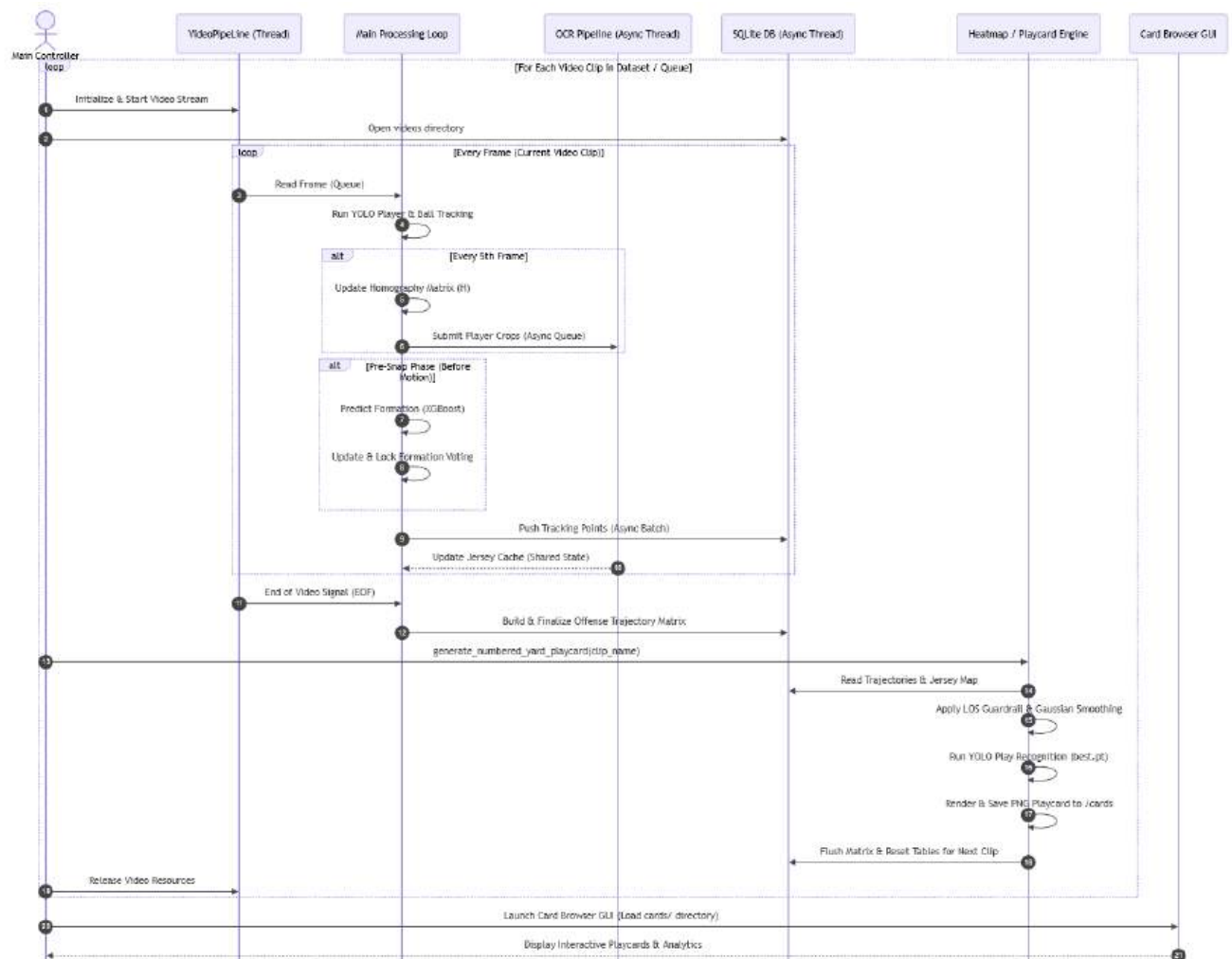


Abbildung 15: UML-Sequenzdiagramm der mehrstufigen, asynchronen Analyse-Pipeline für Football-Videodateien

Das nachfolgende Sequenzdiagramm illustriert den chronologischen Ablauf und das Zusammenspiel der einzelnen Subsysteme innerhalb der Gesamtarchitektur. Die Integration führt die isolierten Module der Objekterkennung, Homographie-Transformation, Trikotnummernerkennung, Formationsklassifizierung sowie Datenbankverwaltung in einer zentralen Pipeline zusammen.

Die Orchestrierung des Gesamtsystems erfolgt über das Hauptskript `main.py`, welches iterativ alle im Zielverzeichnis liegenden Videoclips verarbeitet und dem Benutzer über Interaktionsmöglichkeiten wie das Überspringen einzelner Sequenzen mit der Taste N oder das vorzeitige Abbrechen via Q eine flexible Steuerung bietet. Zum Start einer Analyse-Schleife wird die `VideoPipeLine` in einem eigenen Thread initialisiert, um das Videomaterial frameweise in eine Verarbeitungswarteschlange zu laden. Der Main Processing Loop greift auf diese Frames zu und führt für jedes Einzelbild das YOLO-basierte Spieler- und Ball-Tracking aus. Um Rechenressourcen effizient zu nutzen, wird die Homographie-Matrix zur Projektion der Bounding-Box-Fußpunkte in reale Feldkoordinaten periodisch bei jedem fünften Frame aktualisiert. In diesem Intervall werden zudem die extrahierten Bildausschnitte der Spieler asynchron an die OCR Pipeline übergeben, um die Trikotnummern ohne Blockierung des Hauptthreads zu identifizieren.

Während der zeitkritischen Pre-Snap-Phase vor Beginn des Spielzugs greift der Hauptprozess auf das XGBoost-Modell zu, um die aktuelle Offensiv-Formation vorherzusagen und das Ergebnis über ein internes Voting-System zu sichern. Sämtliche ermittelten Raumkoordinaten werden fortlaufend in einem asynchronen Batch an die SQLite DB übermittelt, welche durch spezialisierte Datenklassen strukturiert verwaltet wird. Parallel dazu meldet die OCR-Pipeline erkannte Trikotnummern zurück, woraufhin der globale Jersey-Cache im Hauptprozess aktualisiert wird. Während der laufenden Verarbeitung werden in der Konsole kontinuierlich relevante Laufzeitmetriken wie die aktuellen FPS, der Clip-Name, die identifizierten Spieler sowie die erkannten Trikotnummern samt zugehöriger OCR-Konfidenzwerte ausgegeben.

Sobald der Videoclip das End-of-File-Signal erreicht, schließt der Main Processing Loop die Generierung der Trajektorien-Matrix ab und übergibt den Steuerbefehl an die Heatmap / Playcard Engine. Diese Komponente liest die aggregierten Trajektorien sowie die Zuordnungstabelle der Trikotnummern aus der SQLite-Datenbank aus, wendet eine Gaußsche Glättung sowie Schutzfilterungen für die Line of Scrimmage an und führt eine abschließende Spielzug-Erkennung durch.

Das Ergebnis wird als visuelle Playcard gerendert und im dedizierten Verzeichnis als PNG-Datei abgespeichert, woraufhin die Datenbanktabellen für den nächsten Clip zurückgesetzt werden. Nach dem Freigeben der Video-Ressourcen durch den Main Controller wird schließlich die Benutzeroberfläche des Card Browser GUI gestartet. Dieser Card-Browser lädt alle generierten Playcards aus dem Verzeichnis und ermöglicht das interaktive Durchmustern und Analysieren der finalen Auswertungsergebnisse.

5. Gesamtevaluation

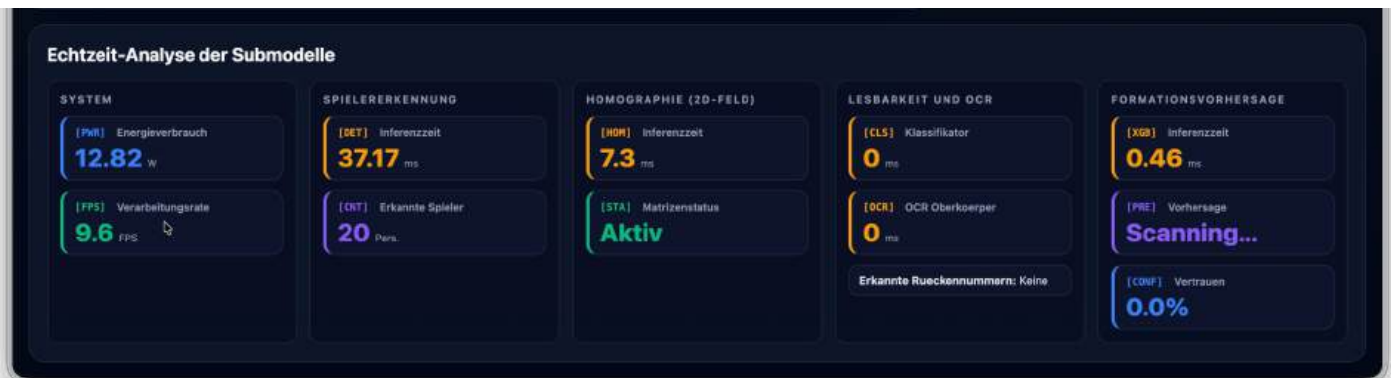


Abbildung 16: Leistungsdaten und Bildraten der Pipeline auf dem NVIDIA Jetson Nano

Die abschließende Evaluierung des Gesamtsystems erfolgt durch die Einordnung der Ergebnisse in den Kontext von Edge Machine Learning. Das primäre Ziel der Entwicklung bestand darin, eine hochleistungsfähige Pipeline zu schaffen, die nicht ausschließlich auf leistungsstarker Server-Hardware operieren kann, sondern auch auf ressourcenbeschränkten Geräten direkte Auswertungen ermöglicht. Um die Flexibilität und Skalierbarkeit der Architektur zu überprüfen, wurde das System über den gesamten Entwicklungszyklus hinweg auf unterschiedlichen Hardware-Plattformen getestet und schrittweise optimiert.

Die Tests zeigten, dass die Pipeline auf Standard-Laptops sowie auf Systemen mit dedizierten NVIDIA RTX Grafikprozessoren hervorragende Durchsatzraten erzielt. Auf diesen modernen Client-Systemen erreicht die Gesamtpipeline flüssige Verarbeitungsgeschwindigkeiten von 25 bis 32 Frames pro Sekunde, was eine vollständige Echtzeitverarbeitung von Videostreams garantiert. Zur Überprüfung der echten Edge-Tauglichkeit wurde nach jedem wesentlichen Entwicklungsschritt eine Portierung auf den NVIDIA Jetson Nano durchgeführt. Auf dieser extrem ressourceneffizienten Edge-Hardware erzielte das System stabile Bildraten von 10 bis 15 Frames pro Sekunde. Ergänzend dazu wurde erfolgreich ein Anwendungsfall demonstriert, bei dem ein Jetson Nano als zentraler Host-Server fungiert, welcher die Analyse-Pipeline ausführt und die Ergebnisse für andere Endgeräte im Netzwerk bereitstellt.

Die Erreichung dieser bemerkenswerten Performanz auf leistungsschwächeren Systemen und sogar auf Low-End-Laptops, auf welchen durchgehend mindestens 10 Frames pro Sekunde gehalten werden konnten, ist maßgeblich auf gezielte architekturelle Optimierungen zurückzuführen. Die Rechenlast wird durch eine zeitliche Selektion kritischer Aufrufe effizient verteilt, indem rechenintensive Operationen wie die Homographie-Berechnung nur jeden fünften Frame ausgeführt werden. Zudem erfolgen aufwendige Nachbereitungen wie die Spielzug-Erkennung sowie das Rendern der visuellen Playcards erst nach dem vollständigen Durchlauf der Video-Sequenz, wodurch die laufende Frame-Verarbeitung entlastet wird.

Theoretisch ermöglicht diese modulare und leichtgewichtige Struktur einen autarken Einsatz des Systems direkt am Spielfeldrand auf kompakter Edge-Hardware. Ein finaler Praxistest unter realen Bedingungen vor Ort steht im Rahmen dieser Arbeit zwar noch aus, die empirisch ermittelten Performance-Daten bestätigen jedoch bereits das hohe Potenzial des EdgeML-Ansatzes für die mobile Football-Analyse.

6. Reflexion

Die Entwicklung des Gesamtsystems zur automatisierten Analyse von Football-Spielen war von wertvollen Erkenntnissen, aber auch von erheblichen technischen und organisatorischen Herausforderungen geprägt. Im Rückblick auf den zehnwöchigen Projektverlauf erwies sich die Bewältigung des Übergangs von statischen Bildmodellen zu einer echtzeitfähigen Video-Processing-Pipeline als die zentralste technische Hürde. Während die isolierte Objekterkennung mittels YOLO bereits in den frühen Iterationen hervorragende Genauigkeitswerte von über 90 Prozent mAP@50 auf Einzelbildern erzielte, offenbarte die praktische Zusammenführung aller Module im fortlaufenden Videostream unvorhergesehene Effekte. Das Phänomen des visuellen Nachziehens, auch bekannt als Ghosting, sowie temporäre ID-Switches bei schnellen Spielerbewegungen zeigten deutlich, dass eine hohe Präzision auf Einzelbildebene nicht automatisch eine fehlerfreie Performanz im dynamischen Videofluss garantiert. Erst durch die Einführung von zeitlichem Pooling, Median-Filterung und asynchroner OCR-Verarbeitung konnte die Trajektorien-Erfassung stabilisiert werden.

Auf Basis dieser Ergebnisse lässt sich die in der Einleitung aufgeworfene zentrale Forschungsfrage positiv beantworten: Die Ergebnisse zeigen klar, dass eine mehrstufige Computer-Vision-Pipeline auch auf ressourcenbeschränkter Edge-Hardware wie dem NVIDIA Jetson Nano betrieben werden kann und dort echtzeitnahe Verarbeitungsgeschwindigkeiten von 10 bis 15 Frames pro Sekunde erreicht. Durch gezielte Optimierungen, wie das Anpassen des Homographie-Intervalls auf jeden fünften Frame sowie das Auslagern rechenintensiver Analysen an das Ende einer Sequenz, konnte die Rechenlast entscheidend gesenkt werden, ohne dass dabei relevante Einbußen bei der visuellen Genauigkeit oder der Erkennungsrate der Offensiv-Formationen in Kauf genommen werden mussten.

Ein weiterer kritischer Punkt lag in den Einschränkungen der genutzten Dateninfrastruktur und der ungleichmäßigen Klassenverteilung im Datensatz. Die anfängliche Beschränkung auf kostenlose Plattformen wie Roboflow mit einem Limit von 600 Bildern machte die Migration der gesamten Trainingsinfrastruktur auf die Hochleistungsrechner der Universität erforderlich. Dieser Schritt ermöglichte zwar den Ausbau auf 1.500 annotierte Bilder sowie das Training komplexerer YOLO-Architekturen, verdeutlichte jedoch auch die organisatorischen Gefahren bei der dezentralen Datenerhebung. So führte die späte Lieferung von redundantem Bildmaterial kurz vor Projektende dazu, dass annotierte Ressourcen verworfen werden mussten, um die Systemstabilität nicht zu gefährden. Zugleich zeigte sich im Modelltraining, dass seltene Objektklassen wie Quarterbacks oder Schiedsrichter durch gezieltes Rebalancing vor einem Overfitting auf die dominanten Feldspielerklassen geschützt werden mussten.

Aus Sicht des Projektmanagements und der Prozess-Retrospektive bot der verordnete Scrum-Ansatz wertvolle Leitplanken, um das komplexe System in klaren meilensteinbasierte Subsysteme wie Homographie, OCR, XGBoost-Klassifikation und Edge-Portierung zu strukturieren. In der studentischen Praxis stieß das theoretische Framework jedoch an deutliche Grenzen. Aufgrund variierender zeitlicher Verfügbarkeiten der Teammitglieder durch Nebenjobs und Prüfungsphasen konnten formelle Scrum-Events wie das tägliche Daily Scrum nicht aufrechterhalten werden und mussten durch flexible Sync-Meetings ersetzt werden. Zudem erforderte eine zeitweise Inaktivität von Teilen des Entwicklungsteams das Aufbrechen der strikten Rollentrennung, sodass Product Owner und Scrum Master aktiv in die technische Umsetzung eingreifen und Aufgaben im Push-Verfahren steuern mussten. Für die finale Projektphase erwies sich der pragmatische Wechsel in eine kaskadierende Entwicklung als überlebenswichtig, um die lauffähige Pipeline pünktlich auf der Jetson-Nano-Hardware bereitzustellen.

Sollte das Projekt in Zukunft weitergeführt werden, stehen primär die Anbindung an Live-Kamerastreams direkt am Spielfeldrand sowie die Steigerung der Gesamteffizienz der Pipeline im Fokus. Hierzu gehört die kontinuierliche Verbesserung und Nachjustierung der KI-Modelle, um verbleibende Latenzen weiter zu reduzieren. In einem nächsten Ausbauschritt soll das funktionale Spektrum erweitert werden, um den Trainern noch tiefergehende analytische Mehrwerte zu bieten. Dies umfasst die Ausweitung des Trackings und der Routen-Nachzeichnung auf die Defensivspieler, die Implementierung eines präzisen Ball-Trackings sowie die automatische Generierung individueller Leistungsstatistiken für einzelne Spieler über den gesamten Spielverlauf hinweg. Trotz der organisatorischen und technischen Hindernisse hat das Projekt erfolgreich demonstriert, dass eine komplexe, mehrstufige Computer-Vision-Pipeline auf kompakter Edge-Hardware betrieben werden kann und als solide Basis für zukünftige sportanalytische Anwendungen dient.

Literaturverzeichnis

- [Route Identification in the National Football League - Chu et al. \(2020\) Link/URL](#)
- [American Football Play Type and Player Position Recognition - Hong et al. \(2024\) Link/URL](#)
- [A General Framework for Jersey Number Recognition in Sports Video - Koshkina et al. \(2024\) Link/URL](#)
- [Player target tracking and detection in football game video using edge computing and deep learning - Jing et al. \(2022\) Link/URL](#)
- [Using Computer Vision and Machine Learning to Automatically Classify NFL Game Film and Develop a Player Tracking System - Ajmeri et al. \(2018\)](#)
- [Roboflow docs](#)
- [Basketball AI: Player Tracking, Team Detection, and Number Recognition with Python](#)
- [Build an AI/ML Football Analysis system with YOLO, OpenCV, and Python](#)
- [YOLO26 on NVIDIA Jetson Setup & Benchmarks | Ultralytics](#)

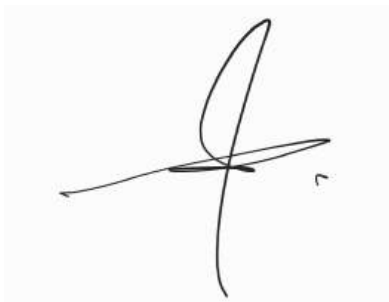
Eigenständigkeitserklärung / Urheberrechtshinweis

Hiermit wird versichert, dass die vorliegende Arbeit sowie alle darin enthaltenen Komponenten einschließlich des Quellcodes, der Datenanalysen, der Evaluierungsergebnisse und sämtlicher Abbildungen zu 100 % das Ergebnis eigenständiger Arbeit der Verfasser sind.

Sämtliche Grafiken, Flussdiagramme, Benchmarks und Screenshots der Benutzeroberfläche stellen ausnahmslos **eigene Darstellungen** dar, die im Rahmen der Entwicklung und Erprobung des Gesamtsystems generiert wurden. Inhalte, Gedanken oder Darstellungen, die direkt oder indirekt aus fremden Quellen übernommen wurden, sind als solche an den entsprechenden Stellen eindeutig gekennzeichnet.

Ort, Datum: Frankfurt am Main, den 05.08.2026

Unterschriften:



Amin Lahseb: 1414161



Ayman Lahseb: 1432554



Juan Fernando Lenis Serna:
1438516



Sahilpreet Singh: 1513808



Muhammad Naeem Abbas: 1469530



Arshad Reyah:
1513428